American University of Phnom Penh

Department of Information Technology

Sothyro Meas (UID:2021320)

Plagiarism Detection in OCR-Extracted Khmer Text: Method Comparison and Web Application Implementation

Final Year Project Report

Supervisor:

Tek Ming Ng, PhD

Co-supervisor:

Aruna Shankar, PhD

Phnom Penh, 25th April 2024

Author's declaration of originality

This report has been prepared on the basis of my own work. Where other published and unpublished source materials have been used; these have been acknowledged.

Student Name: Sothyro MEAS (UID: 2021320)

Date of Submission: 25th April, 2024

Abstract

Detecting plagiarism in the Khmer language remains a significant challenge in Cambodia due to the limited availability of digitized texts. This limitation has led to frequent plagiarism found in many academic papers across institutions, emphasizing the urgent need for a digital solution.

This project aims to develop a plagiarism detection web application specifically for the Khmer language and to identify the most effective approach by comparing four methods: Term Frequency–Inverse Document Frequency with cosine similarity, N-gram matching using a PostgreSQL inverted index with Jaccard similarity, Bidirectional Encoder Representations from Transformers (BERT), and Elasticsearch combined with the Rapidfuzz Python library. The most efficient method is integrated into the web application. Since all collected PDFs are image-based and contain non-selectable text, they are processed using Optical Character Recognition (OCR). The extracted text then undergoes a Khmer-specific data preprocessing pipeline before analysis.

The web application is currently deployed on self-managed servers at the Ministry of Education, Youth and Sport of Cambodia (MoEYS), the primary funder of this initiative. Thirteen universities in Cambodia are using it to assess educational materials in the Khmer language, helping evaluate the system and identify areas for further improvement.

While the application successfully detects identical and similar plagiarized sentences, its accuracy is limited by the quality of the OCR. Common OCR errors often distort textual content, reducing the accuracy of plagiarism detection. Future work will focus on improving OCR performance and incorporating internet-based plagiarism detection to expand the system's capabilities.

List of Abbreviations and Terms

AI	Artificial Intelligence
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
CLPD	Cross Language Plagiarism Detection
DOCX	Microsoft Word Open XML Document Format
FAISS	Facebook Artificial Intelligence Similarity Search
GPU	Graphics Processing Unit
HTML	Hypertext Markup Language
HTTPS	Hypertext Transfer Protocol Secure
IDE	Integrated Development Environment
MoEYS	Ministry of Education, Youth and Sport of Cambodia
NLP	Natural Language Processing
NLTK	Natural Language Toolkit
OCR	Optical Character Recognition
PDF	Portable Document Format
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
TF-IDF	Term Frequency-Inverse Document Frequency
UTF-8	Unicode Transformation Format – 8-bit
XML	Extensible Markup Language

Table of Contents

List of Figures

List of Tables

1. Introduction

Many academic institutions in Cambodia face difficulties in detecting plagiarism in Khmer-language content because the number of available digitized texts remains insufficient. Even when documents are in PDF format, they become non-searchable in Khmer, making automated analysis nearly impossible without OCR. These limitations restrain institutions from reliably assessing originality in Khmer academic documents.

To address these challenges, the Ministry of Education, Youth and Sports (MoEYS) secretariat proposed compiling all documents in digital format and developing a Khmer-language plagiarism-detection tool that extracts text from image-based or non-searchable Khmer PDF documents to deter unethical practices.

This section outlines the problem statement, relevant references, and presents the motivation and background of the project.

1.1 Problem Statement

According to insights shared during a personal interview with the MoEYS secretariat, many authors and researchers in Cambodia have reused redundant content to produce additional publications. This pattern aligns with findings from a 2023 report by the Cambodian Education Forum, which has shown that plagiarism occurs not only among students but also among academic staff [1]. The limited availability of digital Khmer documents, combined with the fact that many academic papers have not been digitized in searchable or highlightable formats, prevents advanced internet-based plagiarism detection tools, such as Grammarly, Chegg, or Turnitin, from working effectively with the Khmer language.

1.2 Motivation

This plagiarism detection web application is designed to provide a solution for Cambodian universities' academic departments, faculty members, document publishers, and educational

institutions to verify original works in the Khmer language by comparing them with previously stored documents in the database. With this tool, institutions will save time and resources by analyzing large documents for plagiarism through a user-friendly web-based interface and receive plagiarism detection results via email. Additionally, this application addresses challenges related to non-digitized hard-copy texts and non-highlightable PDF documents by utilizing Optical Character Recognition (OCR) to convert them into searchable digital text.

1.3 Project Background

Initiated and funded by the Ministry of Education, Youth, and Sport of Cambodia, this project aims to detect plagiarism in academic publications across educational institutions. To enforce research integrity, the Secretary of State at MoEYS has also authorized thirteen public universities in Cambodia to test this platform for reviewing research papers, documents, and published books.

This initiative provides a centralized solution for detecting duplicated content, ensuring that Cambodia's academic works maintain high levels of originality and credibility.

Looking ahead to the future, this project will allow all institutions in Cambodia to use the platform and check for plagiarism easily as digital document datasets will grow every year.

2. Literature Review

This section reviews existing research on Cross-Language Plagiarism Detection (CLPD) using multilingual Bidirectional Encoder Representations from Transformers (BERT) and Khmer semantic search engines. It discusses current limitations and demonstrates how addressing these issues is essential for building an effective plagiarism detection tool for the Khmer language.

2.1 Plagiarism Detection for Under-Resourced Languages

A recent study by Karen Avetisyan explores advancements in Cross-Language Plagiarism Detection (CLPD) using a pre-trained multilingual BERT model, achieving high accuracy for under-resourced languages [2]. The researchers used Armenian as a test language and demonstrated that BERT's contextual embeddings can effectively capture semantic similarities across multiple languages. However, the study has not yet been extended to Southeast Asian languages like Khmer, which have distinct challenges, such as a continuous script and the lack of standardized word segmentation tools. Inspired by this work, this project will also utilize this BERT-based technique specifically for the Khmer language, aiming to address its structural complexities and evaluate the model’s effectiveness on Khmer academic content.

2.2 Khmer Semantic Search Engine

Another study by Nimol Thuon on a Khmer semantic search engine utilizing Term Frequency-Inverse Document Frequency (TF-IDF) focuses on enhancing document retrieval based on keyword searches in the Khmer language [3]. The system also allows users to upload documents to the database and search for similar content within the uploaded files. While the system effectively retrieves relevant content, its functionality is limited to sentence or keyword-based searching and does not support full document upload of PDF or Word files for plagiarism analysis. This limitation highlights the need for a more comprehensive approach to Khmer text analysis that includes OCR and plagiarism detection across various document formats.

3. Project Objective, Deliverables and Supporting Organization

This section outlines the expected outcomes of the project and the key components that need to be developed to deliver an efficient plagiarism detection web application for the target users. The system was developed in alignment with the needs of MoEYS and thirteen universities in Cambodia. Their involvement ensured that the final platform is both relevant and usable within the MoEYS and across the participating universities in Cambodia.

3.1 Main Objective and Key Deliverables

The main objective of this project is to develop a Khmer language plagiarism detection system that allows educational institutions, publishers, and researchers in Cambodia to verify content originality. This system allows users to upload documents and compare new submissions against previously uploaded content. Moreover, it also utilizes OCR to extract Khmer text from PDF files, enabling the upload of non-highlightable formats. The objectives of this project include:

1. Data gathering, cleaning, and pipeline development
    - Compile hundreds of sample documents in Microsoft Word and PDF formats provided by the Cambodian Ministry of Education, Youth, and Sports (MoEYS).
    - Preprocess each document by text cleaning and tokenization.
    - Create metadata (e.g., document ID, title, university) for each document, and store it in both the database and the file storage system.
2. Integrating Optical Character Recognition (OCR) for non-highlightable documents
    - Convert all PDF pages to images.
    - Extract text from each image using OCR to enable plagiarism detection.
    - Identified the most accurate OCR method for text extraction
3. Developing a scalable plagiarism detection functionality
    - Identified the fastest, most accurate, and scalable similarity search method to detect plagiarized content within a large academic dataset.

    - Integrate the best plagiarism detection method within the web application.
4. Providing a user-friendly web-based platform
    - Develop a web interface that allows users to upload documents for plagiarism detection.
    - Deliver plagiarism detection results (e.g., similarity score, matching reference sentences from the database, and associated university names) in downloadable Excel files via email.
    - Allow users to view, delete, or download all uploaded files in the database as needed.
    - Display real-time storage usage and alert users when storage is full.
    - Provide login access for admin and standard users.
5. Deployment on a self-managed server at the Cambodian Ministry of Education, Youth, and Sports
    - Host the file storage system and web application on a self-managed server at MoEYS.
    - Allow all authorized users to access the web application via the domain name provided by MoEYS (domain name: plagiarism-checker.duraseksa.gov.kh)

3.2 Supporting Organization and Collaboration

The learning material design team from the Center for Digital Distance Learning at MoEYS provided the document dataset and defined the system requirements to ensure the platform meets the practical needs of Cambodian educational institutions. In addition, the MoEYS technical team also supported the deployment of the web application on the ministry's self-managed server.

4. Methodology and Implementation

This section presents the methodology and technical implementation of the Khmer-language plagiarism detection system. It describes the methods evaluated and identifies the most suitable approach for integration into the web application. Each method follows the same workflow used to assess its effectiveness: preparing the input data, applying the detection technique, and evaluating the output results.

The section begins with an overview of the development environment and the tools employed. It then details the data preparation process used to extract clean Khmer text as input for four plagiarism detection approaches: TF-IDF with cosine similarity, an N-gram inverted index with Jaccard similarity implemented in PostgreSQL, BERT embeddings with FAISS indexing, and Elasticsearch.

Each method is examined in terms of its concepts, technical implementation, and performance limitations. Finally, the results are compared to assess which techniques are most effective for plagiarism detection in the context of the Khmer language.

4.1 Development Environment and Technology Stack

The development environment consists of the software frameworks, tools, and services used to develop the Khmer-language plagiarism detection system. It covers backend and frontend development, OCR and text extraction, machine learning tools, storage systems, and deployment. The technologies are grouped as follows:

- Machine Learning and NLP Tools
    - Scikit-learn – Implements TF-IDF vectorization and cosine similarity.
    - Transformers (Hugging Face) – Generates Khmer language text embeddings using the multilingual BERT model.
- Optical Character Recognition and PDF Text Extraction Tools
    - Tesseract OCR – Extracts text from image-based documents.
    - Google Cloud Vision AI – Provides Google Cloud-based OCR for text extraction.

    - Pdfplumber, Pdfminer, Fitz – Extracts text from digital PDF files using Python libraries.
- Search and Retrieval Tools
    - Elasticsearch – Supports similarity search and large-scale text retrieval.
    - FAISS – Enables fast vector similarity search for BERT-based embeddings.
- Storage and Background Task Management
    - MinIO – Stores uploaded documents and supports secure file backup.
    - Redis – Manages background tasks and job queues for asynchronous plagiarism detection and document uploading.
- Programming Languages and Frameworks
    - Python / Flask – Develops backend and logic.
    - React.js – Develops a responsive and interactive frontend interface.
- Deployment Tools
    - MoEYS Server – Serves as the production hosting environment.
    - Docker – Containerizes and manages all services for consistent deployment.
- Development tools
    - Visual Studio Code– Used for source code development.
    - Postman – Used for testing API endpoints.
    - Jupyter Notebook – Used for testing and evaluating plagiarism detection methods.

4.2 Data Preparation

This section outlines the steps taken to prepare data for plagiarism detection, including data acquisition, file type parsing, and preprocessing. In the data acquisition phase, documents are provided by official sources from the MoEYS. Since the system accepts two file formats, PDF and Microsoft Word documents, each file is processed according to its type. Once the raw text is extracted, it is cleaned and tokenized using a custom Khmer language preprocessing pipeline. This same pipeline is applied to both documents uploaded by users and those being checked for plagiarism, ensuring consistency.

4.2.1 Data Acquisition

The dataset used in this project was provided via Google Drive by the MoEYS and includes a total of 900 books collected from thirteen universities across Cambodia. These documents cover a wide range of topics, including agriculture, technology, literature, and more. Among these, 890 files are in PDF format, while the remaining 10 are in Microsoft Word format. Since all of the PDF files are non-highlightable or image-based, Optical Character Recognition (OCR) is required to extract readable text before any plagiarism detection methods can be applied.

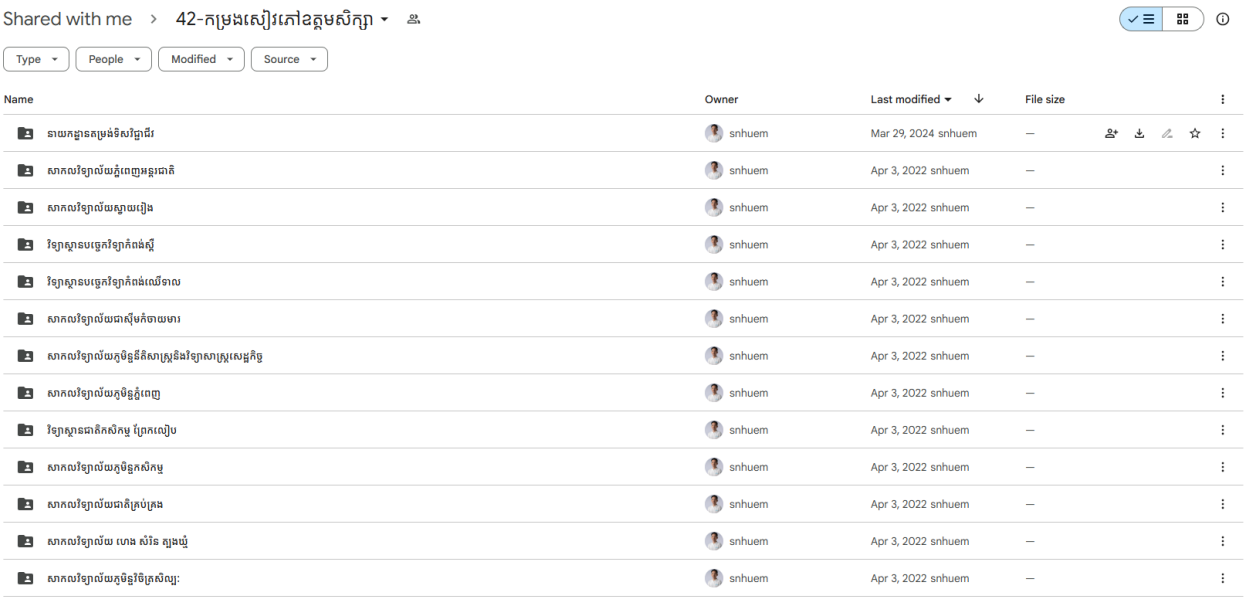


Figure 1: Dataset Collection

4.2.2 Different File Type Parsing

This section outlines how different file types are handled during the parsing phase to ensure consistent text extraction. Each format requires a specific processing method, for instance, PDF files, are processed using Optical Character Recognition (OCR) to extract text, while Microsoft Word documents are parsed directly using Python library called *python-docx* [4].

4.2.2.1 Portable Document File Parsing

Several Python libraries were explored for extracting text directly from PDF files, including PyMuPDF, Pdfminer, and pdfplumber. Although these tools are widely used for extracting text from PDF files, they do not produce accurate results when applied to Khmer text.

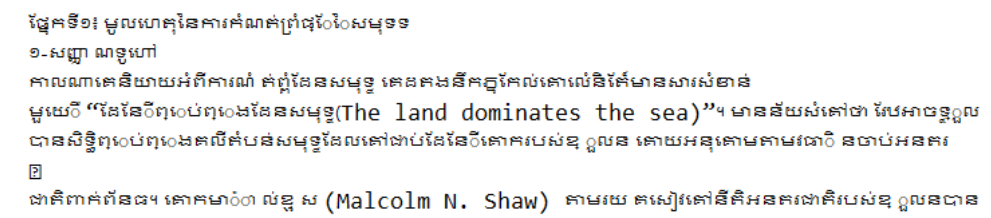
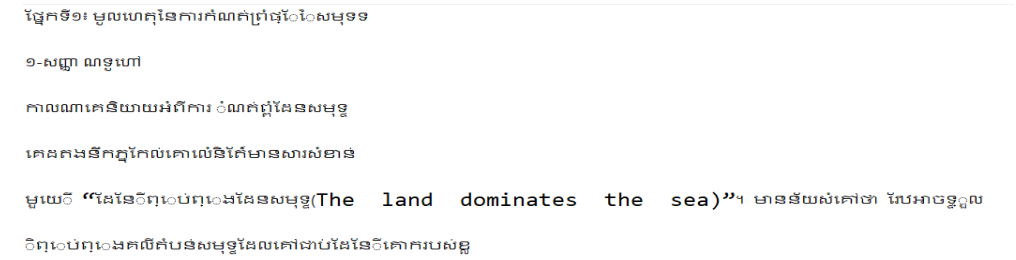

ផ្នែកទី១៖ មូលហេតុនៃការកំណត់ព្រំដែន
១-សញ្ញា ណទូហៅ
កាលណាគេនិយាយអំពីការណ៍ ត់ព្ដំដែនសមុទ្ទ គេឧតងនឹកក្ដកែល់គោលេ៌និតែ៌មានសារសំខាន់
មួយេី "ដែនែីពុេប់ពុេងដែនសមុទ្ទ(The land dominates the sea)"។ មានន័យសំគៅថា រែេអាចទ្ទួល
បានសិទ្ធិពុេប់ពុេងគលីតំបន់សមុទ្ទដែលគៅជាប់ដែនែីគោករបស់ខ្ួ ួលន គោយអនុគោមគាមរថាិ នថាប់អនគរ
ជាគិពាក់ព័នធ។ គោកមាច់ថា ល់ខ្ួ ស (Malcolm N. Shaw) គាមរយ គសៀវគៅនីគិអនគរជាគិរបស់ខ្ួ ួលនបាន

Figure 2: Text Extraction with Pdfplumber


ផ្នែកទី១៖ មូលហេតុនៃការកំណត់ព្រំដែន

១-សញ្ញា ណទូហៅ

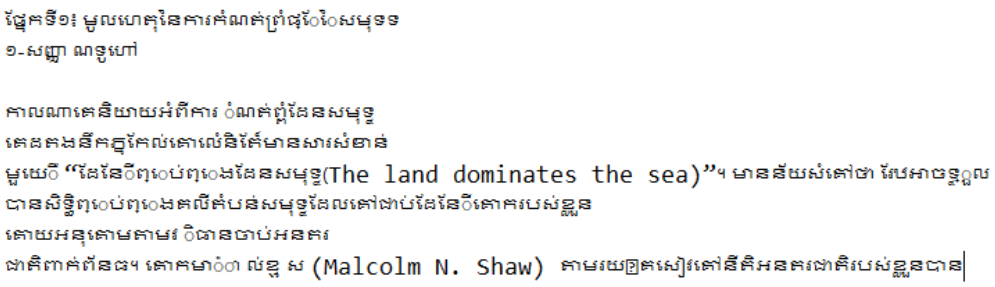
កាលណាគេនិយាយអំពីការ ំណាត់ព្ដំដែនសមុទ្ទ

គេឧតងនឹកក្ដកែល់គោលេ៌និតែ៌មានសារសំខាន់

មួយេី "ដែនែីពុេប់ពុេងដែនសមុទ្ទ(The land dominates the sea)"។ មានន័យសំគៅថា រែេអាចទ្ទួល

ិពុេប់ពុេងគលីតំបន់សមុទ្ទដែលគៅជាប់ដែនែីគោករបស់ខ្ួ

Figure 3: Text Extraction with Pdfminer


ផ្នែកទី១៖ មូលហេតុនៃការកំណត់ព្រំដែន
១-សញ្ញា ណទូហៅ

កាលណាគេនិយាយអំពីការ ំណាត់ព្ដំដែនសមុទ្ទ
គេឧតងនឹកក្ដកែល់គោលេ៌និតែ៌មានសារសំខាន់
មួយេី "ដែនែីពុេប់ពុេងដែនសមុទ្ទ(The land dominates the sea)"។ មានន័យសំគៅថា រែេអាចទ្ទួល
បានសិទ្ធិពុេប់ពុេងគលីតំបន់សមុទ្ទដែលគៅជាប់ដែនែីគោករបស់ខ្ួន
គោយអនុគោមគាមរ ិធានថាប់អនគរ
ជាគិពាក់ព័នធ។ គោកមាច់ថា ល់ខ្ួ ស (Malcolm N. Shaw) គាមរយគសៀវគៅនីគិអនគរជាគិរបស់ខ្ួនបាន

Figure 4: Text Extraction with PyMuPDF (Fitz)

We also installed Tesseract OCR for the Khmer language on our Windows environment by downloading the custom Khmer-trained data file from the Tesseract OCR GitHub repository [5]. Once installed, Tesseract was used to extract text from PDF files; however, each page had to be converted into an image using the *pdf2image* function before running OCR, since it doesn't support direct PDF text extraction. The extracted text was still not fully accurate, as shown in the following figure, and the preprocessing workflow proved inefficient because the entire PDF had to be converted into images before text extraction, which consumed a significant number of resources during the OCR process.

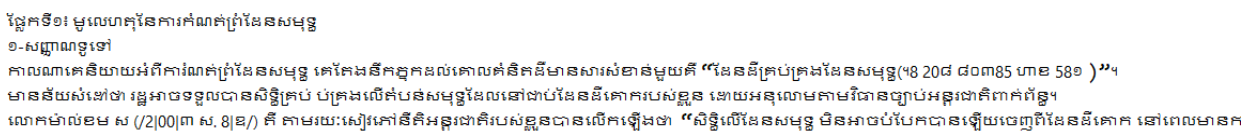

ផ្នែកទី១៖ មូលហេតុនៃការកំណត់ព្រំដែនសមុទ្ធ
១-សញ្ញាណទូទៅ
កាលណាគេនិយាយអំពីការំណត់ព្រំដែនសមុទ្ធ គេតែងនឹកភ្ជុកដល់គោលគំនិតដ៏មានសារសំខាន់មួយគឺ “ដែនដីគ្រប់គ្រងដែនសមុទ្ធ(ឯ8 20ដ ដ০៣85 ហាខ 58១)”។
មានន័យសំដៅថា រដ្ឋអាចទទួលបានសិទ្ធិគ្រប់ ប់គ្រងលើតំបន់សមុទ្ធដែលនៅជាប់ដែនដីគោករបស់ខ្លួន ដោយអនុលោមតាមវិធានច្បាប់អន្តរជាតិពាក់ព័ន្ធ។
លោកម៉ាល់ខម ស (/2|00|៣ ស. 8|ខ/) គឺ តាមរយៈសៀវភៅនីតិអន្តរជាតិរបស់ខ្លួនបានលើកឡើងថា “សិទ្ធិលើដែនសមុទ្ធ មិនអាចប់បែកបានឡើយចេញពីដែនដីគោក នៅពេលមានក

Figure 5: Text Extraction with OCR (Tesseract)

For better efficiency and extraction accuracy, we used Google Cloud Storage to store the PDF files and used the file ID from Google Cloud Storage to process them with Google Cloud Vision OCR. The results were noticeably better than the previous methods, so we decided to use Google Cloud Vision to support the text extraction process.

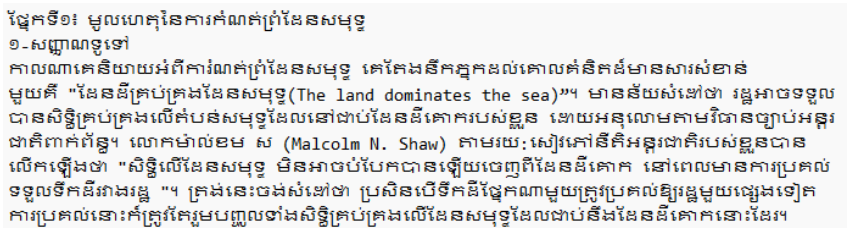

ផ្នែកទី១៖ មូលហេតុនៃការកំណត់ព្រំដែនសមុទ្ធ
១-សញ្ញាណទូទៅ
កាលណាគេនិយាយអំពីការំណត់ព្រំដែនសមុទ្ធ គេតែងនឹកភ្ជុកដល់គោលគំនិតដ៏មានសារសំខាន់
មួយគឺ "ដែនដីគ្រប់គ្រងដែនសមុទ្ធ(The land dominates the sea)"។ មានន័យសំដៅថា រដ្ឋអាចទទួល
បានសិទ្ធិគ្រប់គ្រងលើតំបន់សមុទ្ធដែលនៅជាប់ដែនដីគោករបស់ខ្លួន ដោយអនុលោមតាមវិធានច្បាប់អន្តរ
ជាតិពាក់ព័ន្ធ។ លោកម៉ាល់ខម ស (Malcolm N. Shaw) តាមរយៈសៀវភៅនីតិអន្តរជាតិរបស់ខ្លួនបាន
លើកឡើងថា "សិទ្ធិលើដែនសមុទ្ធ មិនអាចប់បែកបានឡើយចេញពីដែនដីគោក នៅពេលមានការប្រគល់
ទទួលទឹកដីរវាងរដ្ឋ "។ ត្រង់នេះចង់សំដៅថា ប្រសិនបើទឹកដីផ្នែកណាមួយត្រូវប្រគល់ឱ្យរដ្ឋមួយផ្សេងទៀត
ការប្រគល់នោះកំត្រូវតែរួមបញ្ចូលទាំងសិទ្ធិគ្រប់គ្រងលើដែនសមុទ្ធដែលជាប់នឹងដែនដីគោកនោះដែរ។

Figure 6: Text Extraction with Google Cloud Vision AI OCR

4.2.2.2 Microsoft Word Document Parsing

To extract text from Microsoft Word documents in .docx format, we used the Python library *python-docx*, which provides access to the document's structure, including paragraphs and tables. Using this library, the system iterates through all elements in the document body and retrieves

the available text content for further processing. The following pseudocode illustrates the process of extracting text elements from a .docx file using *python-docx*:

Algorithm: Pseudocode for Extracting Text from a Word Document

```
Input: file_path – path to the .docx file
Output: text_content – extracted document text
Open the Word document from the given file path;
Initialize an empty list text_content;
for each block in the document body do
    if block is a paragraph then
        Append the paragraph text to text_content;
    end
    else if block is a table then
        for each row in the table do
            Initialize an empty list row_data;
            for each cell in the row do
                Extract the text and append it to row_data;
            end
            Join row_data with a separator and append to
             text_content;
        end
    end
end
Return text_content;
```

Initially, an empty list called *document_content* is created to store the extracted text. The system then iterates through each element in the document body using the *iterchildren* function from the *python-docx* library. If the element is a paragraph (i.e., its XML tag ends with 'p'), the text is extracted and added directly to the *document_content* list. If the element is a table (tag ends with 'tbl'), the system iterates through each row and collects text from all cells. The cell contents are then joined using the pipe symbol (|) as a separator to form a single formatted row, which is appended to the *document_content* list. This process continue until there is no further content in the document body.

4.2.3 Data Preprocessing Pipeline

After the text is extracted, a series of preprocessing steps are applied to clean and prepare it for plagiarism detection. The raw text often contains inconsistent line breaks, extra spaces,

unnecessary characters, and irregular newlines. Therefore, the following steps are applied to ensure text consistency.

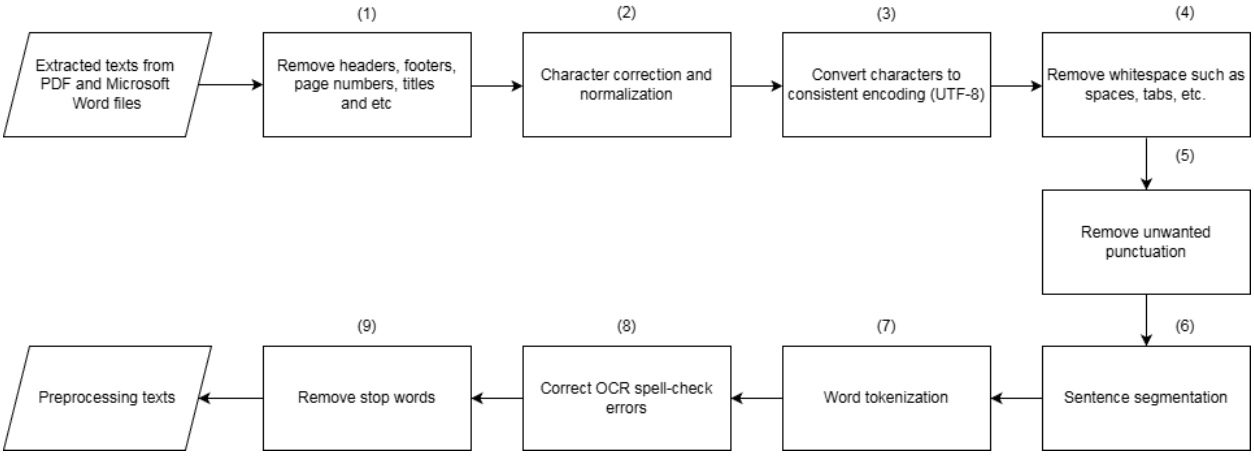


Figure 7: Data Preprocessing Pipeline

1. Remove headers, footers, page numbers, and titles: The raw extracted text often contains non-content elements such as page numbers, document headers and footers, and chapter titles. These are considered textual noise and must be removed to improve the accuracy of plagiarism detection. To filter them out, we observed the following patterns:
    - Lines consisting only numbers with symbols are page numbers.
    - Repeated lines across multiple pages are headers or footers.
    - Lines that follow a numbering format (e.g., "1.", "1.1") and have fewer than 20 characters are removed using regular expressions (regex), as they are titles.
2. Character correction and normalization:
    - Reordered characters for consistency: In Khmer, certain characters can appear visually correct but are actually in different Unicode sequences due to incorrect ordering. For example, the word "ស្ត្រី" (female) can be encoded as either ស + ◌្ + រ + ◌្ + ត + ◌ី or ស + ◌្ + ត + ◌្ + រ + ◌ី. Although these variants look similar on screen, they produce different underlying character sequences, which can negatively affect text comparison [6]. To address this issue, the system ensures that the

character ◌ is always placed between the diacritic marks (◌) to maintain consistent and correctly ordered Unicode representations [7].

- Incorrect character replacement: Some characters may be incorrectly used or misrecognized by OCR. For example, "ស្តី" (scold) should be represented as ស + ◌ + ដ + ◌, but an OCR error most certainly produces ស + ◌ + ត + ◌, substituting the correct consonant (ដ) with a similar-looking but incorrect one (ត). To solve this issue, we can use the replace function in Python to always convert words that consist of ◌ + ត to return ◌ + ដ. [6]

3. Convert characters to consistent encoding (UTF-8): All content is encoded in UTF-8 to ensure consistent character representation.

4. Remove spaces: Extra spaces, tabs, and line breaks are stripped to ensure clean texts.

5. Remove unwanted punctuation: Punctuation that does not contribute to meaning such as ។, ។, “, «, ^,”, (, /,), etc. are removed.

6. Sentence segmentation: The cleaned text is then divided into sentence-level units using Khmer ending punctuation and symbols such as: ។, ៖, ! and ?.

7. Word tokenization: For Khmer, a language that does not use spaces between words, word tokenization is a critical preprocessing step. Methods like TF-IDF, N-gram, and Elasticsearch rely on clearly defined word boundaries to compute meaningful terms. To achieve this, we use the *khmercut* library to segment sentences into smaller word units [8].

8. Correct OCR spell-check errors: OCR can introduce spelling errors, especially with the Khmer script. This step identifies and corrects OCR spelling mistakes to improve

accuracy in plagiarism detection. To implement this, we use a tool provided by Khmerlang spell check correction API to process the tokenized words [9].

Figure 8: OCR Spelling Errors Correction with Khmerlang API

As shown in the above figure, the API returns results that allow us to identify words with OCR errors. We then use the suggestions from the API response to replace those words and reduce errors caused by OCR.

9. Remove stop words: Common but semantically weak words such as “គាត់”, “ការ” or “នេះ” are removed to improve search relevance. To implement this, we have used the Khmer stop word from the Khmer Natural Language Processing community to filter all of the unnecessary words [10].

After preprocessing, the cleaned and standardized text is ready for analysis. This processed data is then used as input across the following plagiarism detection methods. Each method is described in the following sections along with its implementation, and evaluation results.

4.3 Plagiarism Detection Methods

This section outlines different methods that can be used to detect plagiarized content in the preprocessed data. After exploring each method, the following section will evaluate which one offers the best results, and that method will be integrated into our web application system. Each method uses a distinct strategy to measure text similarity, including vector-based approaches like TF-IDF with cosine similarity; set-based techniques like n-gram and Jaccard similarity (stored in

an inverted index table in PostgreSQL); advanced retrieval engines like Elasticsearch; and deep learning models such as BERT combined with FAISS for fast similarity matching.

The following subsections explain each method in detail, including its conceptual overview, technical implementation aligned with system requirements, results, and limitations.

4.3.1 TF-IDF and Cosine Similarity

Term Frequency-Inverse Document Frequency (TF-IDF) and cosine similarity are widely used methods for measuring textual similarity. They convert each sentence into a vector based on word importance and calculate similarity using the cosine of the angle between the two vectors. This section explains how TF-IDF works, its technical implementation, performance, results and limitation.

4.3.1.1 Method Overview

Term frequency (TF) measures how frequently a term appears in a document, normalized by the total number of terms in that document.

$$TF\ (t, d)\ = \frac{Number\ of\ times\ term\ t\ appears\ in\ document\ d}{Total\ number\ of\ terms\ in\ doument\ d}$$

However, Term Frequency alone is not sufficient to determine whether each document is similar because there could be common terms that may appear frequently in many documents but carry no meaning which lead to false positive. Therefore, we combine Inverse Document Frequency to also measure how rare a term is across all documents.

$$IDF\ (t)\ = log\ \frac{Total\ number\ of\ document}{Number\ of\ documents\ containing\ term\ t}$$

If a term appears in many documents, the denominator becomes large, and the overall fraction becomes small. By taking the logarithm of a small number, the IDF score becomes low, making the frequent words appear to be less considered and prioritized words with rarity across the entire document. By multiplying TF and IDF, we ensure a term receives a high TF-IDF score only when it is both frequent and also rare in the overall dataset. [11]

$$TF - IDF\ (t, d)\ =\ TF(t, d)\ \times\ IDF\ (t)$$

To apply this method in our plagiarism detection tool, we first compute the TF-IDF scores for all terms (words) across both the original documents in the database and the newly uploaded document that the user wants to check for plagiarism. These scores are combined into vectors, with each sentence represented as a vector.

After converting each sentence into a TF-IDF vector, we compare the sentence vectors from the newly uploaded document with those from the existing documents in the database. To measure how similar two sentences are, we apply cosine similarity, which calculates the angle between two vectors in a high-dimensional space. The formula for cosine similarity between two vectors, A and B, is shown below

$$cosine\ similarity\ score\ (A, B)\ =\ \frac{A\ .\ B}{\|A\|\|B\|}$$

The smaller the angle, the more similar the sentences are. A cosine similarity scores close to 1 indicates high similarity, while a score near 0 suggests little to no similarity. [11]

4.3.1.2 Method Implementation Process Flow

TF-IDF vectors are computed based on term frequencies and document frequencies across the entire dataset, including both the documents in the database and the documents that the user

wants to check for plagiarism. This process is computationally expensive and inefficient, especially when dealing with many documents.

To address this limitation, we can compute the TF-IDF vectors once for all of the provided documents from MoEYS and utilize the *joblib* library to save the results. *Joblib* is a Python library that efficiently saves and loads large objects like TF-IDF models and vectors to avoid recomputing them repeatedly [12]. Later, when a user uploads a new document to check for plagiarism, we load the saved results and compute only the uploaded document into TF-IDF vectors. These vectors are then compared to each vector in the precomputed matrix using cosine similarity to produce similarity scores. This approach avoids redundant preprocessing by saving the results once and loading them when needed. The diagram below shows the implementation of this method:

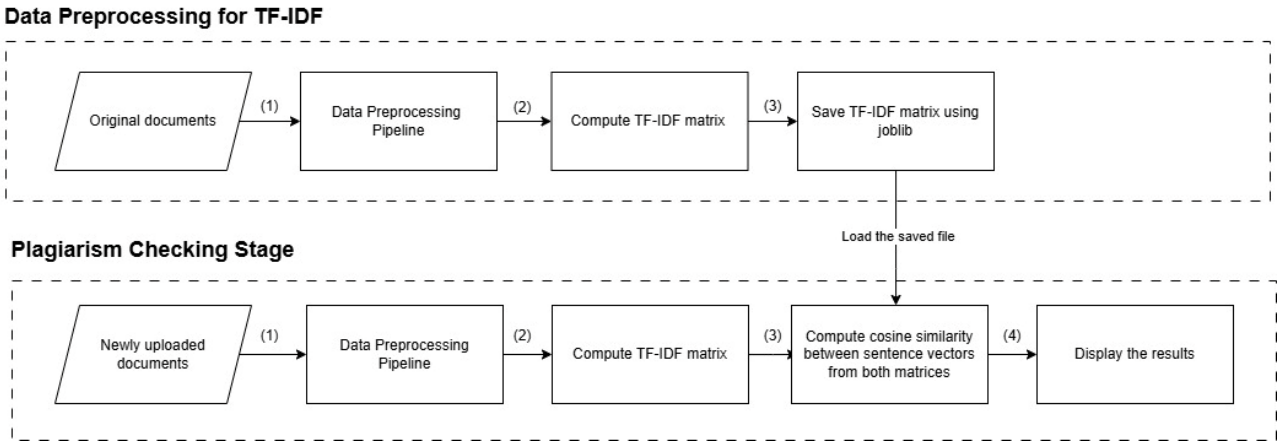


Figure 9: TF-IDF Pipeline Implementation

4.3.1.3 Technical Implementation

For this implementation, we utilize TfidfVectorizer function from Scikit-learn feature extraction library to calculate TF-IDF score and convert each sentence into vector. We divide the implementation into four main stages:

1. Preprocessing and sentence extraction: We load our entire dataset, which contains metadata such as document ID, university, title, and a list of sentences. Each sentence is

converted to TF-IDF vector and mapped to its corresponding metadata, allowing us to trace any detected similarity back to the original source.

2. TF-IDF vectorization and joblib caching: After computing TF-IDF matrix with Scikit-learn, we use the joblib from the Python library to save the vectorizer training model, TF-IDF matrix of the original documents and its metadata. This allows us to avoid computing TF-IDF for the entire original documents in the database again every time a user upload a document to check for plagiarism.

```
from sklearn.feature_extraction.text import TfidfVectorizer
import joblib

vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(all_sentences)

joblib.dump(vectorizer, "vectorizer.pkl")
joblib.dump(tfidf_matrix, "tfidf_matrix.pkl")
joblib.dump(index_to_metadata, "metadata.pkl")
```

3. Transforming uploaded documents: When a user uploads new documents for plagiarism checking, the documents are passed through the same data preprocessing pipeline and transform function from Scikit-learn using the saved TfidfVectorizer from joblib.

4. Measure textual similarity: Cosine similarity is used to compare each uploaded sentence vector with all original sentences vector in order to measure the similarity.

```
def cosine_similarity(vec1, vec2):
    dot_product = sum(a * b for a, b in zip(vec1, vec2))
    magnitude1 = math.sqrt(sum(a ** 2 for a in vec1))
    magnitude2 = math.sqrt(sum(b ** 2 for b in vec2))
    if magnitude1 == 0 or magnitude2 == 0:
        return 0.0
    return dot_product / (magnitude1 * magnitude2)
```

4.3.1.4 Results and Limitations

A key limitation of this method lies in its scalability: frequently adding new documents to the original dataset would require retraining the TF-IDF vectorizer on the entire corpus, which becomes increasingly time-consuming as the dataset grows.

4.3.2 N-gram, PostgreSQL Inverted Index and Jaccard Similarity

N-gram and Jaccard similarity are another technique for comparing sets of text based on shared sequences. N-gram splits text into sequences of *n* words, while Jaccard similarity measures the overlap between two texts. This section outlines how n-gram and Jaccard similarity works, along with their technical implementation to meet the requirements of our plagiarism detection system, their output results, and their limitations [13].

4.3.2.1 N-gram Tokenization

N-gram is a sequence of terms extracted from a given text. In our plagiarism detection tool, we use n-gram to break sentence into smaller segments and identify overlapping segments between sentences.

N-gram on Khmer Language

N = 1	N = 2	N = 3
គាត់	គាត់ទៅ	គាត់ទៅសាលា
ទៅ	ទៅសាលា	
សាលា		

Figure 10: Khmer Language N-Gram

4.3.2.2 Sentence Similarity using Jaccard

Jaccard similarity is a method for comparing the similarity between two sets. In this context, let set A represent the n-grams of the original documents, and set B represent the n-gram of the uploaded document being checked for plagiarism. The Jaccard similarity between those two sets is defined as:

$$Jaccard\ Similarity\ (A, B) = \frac{|A \cap B|}{|A \cup B|}$$

The Jaccard similarity ranges from 0 to 1, where a higher similarity score indicates more overlapping n-grams between the two sets, while a lower score indicates less overlap.

4.3.2.3 Method Implementation Process Flow

Calculating Jaccard similarity based on shared n-grams between all sentence pairs can be memory and computationally expensive, especially when we have a large number of documents. To address this, we use an inverted index method. Each n-gram from every sentence in the original documents is stored in the database, and mapped to the sentence IDs where it appears.

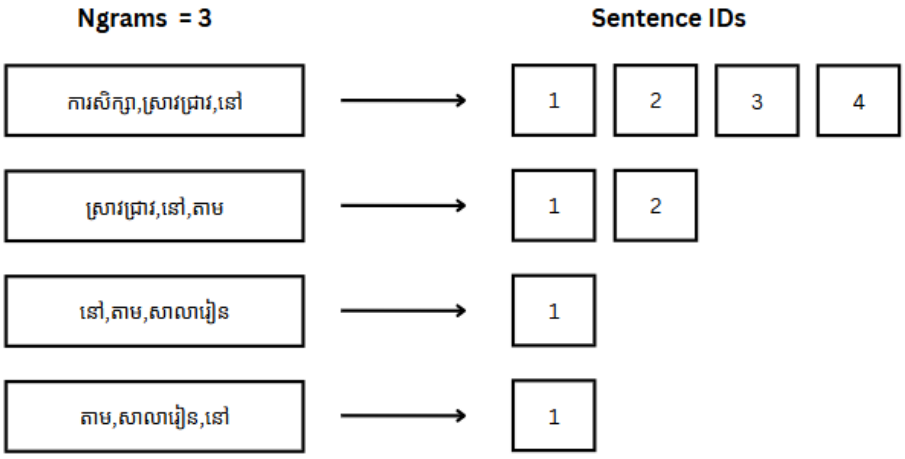


Figure 11: Inverted Index for N-Gram

During the plagiarism detection process, the newly uploaded document is split into sentences, and each sentence is broken into n-grams. These n-grams are then used to search for matches in the n-gram table in the database, allowing the system to quickly retrieve the sentence ID and its full sentence from the original document if they share at least one n-gram. Jaccard similarity is computed only between these candidate sentences, rather than performing a full sentence-by-sentence comparison. The diagram below illustrates the implementation of this method:

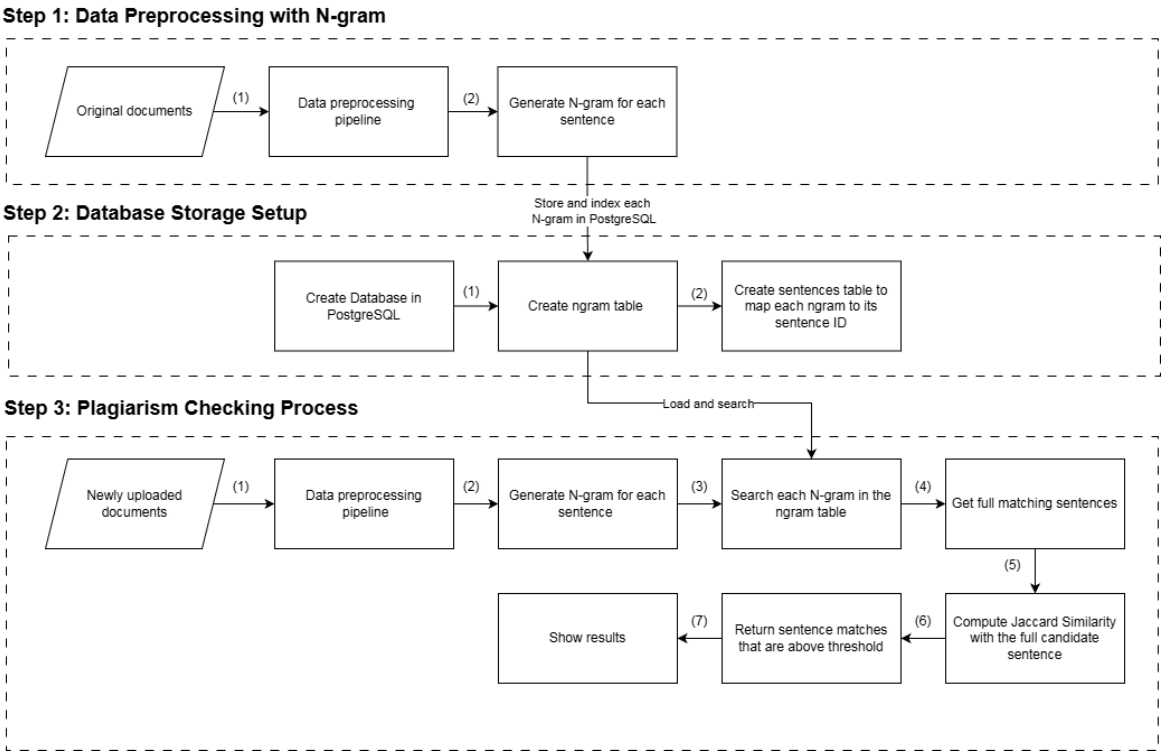


Figure 12: N-Gram, Jaccard Similarity, and PostgreSQL Inverted Index Method Implementation

4.3.2.4 Technical Implementation

First, we iterate through each sentence and its word tokens to create n-gram sequences. These unique n-grams are then indexed in a PostgreSQL database to support fast lookups. When a user uploads a document for plagiarism checking, we extract its n-grams and compare them against the indexed n-grams to retrieve candidate matches and then apply Jaccard to compute the similarity score. The overall implementation is divided into three main stages:

1. Each sentence is processed by splitting it into a list of words, then iterating through the list to group every *n* consecutive word into n-grams. In the following example, we use $n = 3$.

[('លោក', 'ប្រធាន', 'អនុសាខា'), ('ប្រធាន', 'អនុសាខា', 'ជាន'), ('អនុសាខា', 'ជាន', 'បន្ត'), ('ជាន', 'បន្ត', 'ទៀត'), ('បន្ត', 'ទៀត', 'ថា'), ('ទៀត', 'ថា', 'ជានិច្ចកាល'), ('ថា', 'ជានិច្ចកាល', 'សម្តេច'), ('ជានិច្ចកាល', 'សម្តេច', 'តែងតែ'), ('សម្តេច', 'តែងតែ', 'យកចិត្តទុកដាក់'), ('តែងតែ', 'យកចិត្តទុកដាក់', 'ខ្ពស់'), ('យកចិត្តទុកដាក់', 'ខ្ពស់', 'ចំពោះ'), ('ខ្ពស់', 'ចំពោះ', 'សុខទុក្ខ'), ('ចំពោះ', 'សុខទុក្ខ', 'របស់'), ('សុខទុក្ខ', 'របស់', 'បងប្អូន'), ('របស់', 'បងប្អូន', 'ប្រជាពលរដ្ឋ'), ('បងប្អូន', 'ប្រជាពលរដ្ឋ', 'រងគ្រោះ'), ('ប្រជាពលរដ្ឋ', 'រងគ្រោះ', 'ទឹង'), ('រងគ្រោះ', 'ទឹង', 'ងាយ'), ('ទឹង', 'ងាយ', 'រងគ្រោះ'), ('ងាយ', 'រងគ្រោះ', 'គ្រប់'), ('រងគ្រោះ', 'គ្រប់', 'រូប'), ('គ្រប់', 'រូប', 'ដោយ'), ('រូប', 'ដោយ', 'មិន'), ('ដោយ', 'មិន', 'ប្រកាន'), ('មិន', 'ប្រកាន', 'វណ្ណៈ'), ('ប្រកាន', 'វណ្ណៈ', 'ពណ៌'), ('វណ្ណៈ', 'ពណ៌', 'សម្បុរ'), ('ពណ៌', 'សម្បុរ', 'ដំនើរ'), ('សម្បុរ', 'ដំនើរ', 'សាសនា'), ('ដំនើរ', 'សាសនា', 'ឬ'), ('សាសនា', 'ឬ', 'និន្នាការ'), ('ឬ', 'និន្នាការ', 'ទយោបាយ'), ('និន្នាការ', 'ទយោបាយ', 'ណាមួយ'), ('ទយោបាយ', 'ណាមួយ', 'ឡើយ')]

Figure 13: Trigram Output from Sample Khmer Texts

2. We created three tables such as sentences, inverted index and ngrams in PostgreSQL. The sentences table stores each sentence along with its associated metadata, including the document ID, sentence ID (as the primary key), full sentence, author, title, university, and year of publication. The ngrams table stores each unique n-gram from all sentences, with ngram ID as the primary key. The inverted index table maps each n-gram to the sentence it appears in by storing both sentence ID and ngram ID as foreign keys. This structure enables better retrieval of candidate sentences for Jaccard similarity comparison.

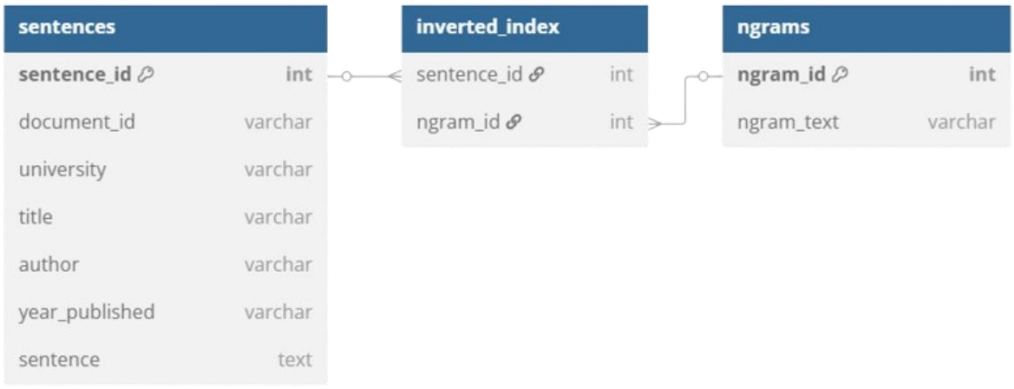


Figure 14: Inverted Index Schema for N-Gram in PostgreSQL

3. Once all candidate sentence IDs are obtained, we retrieve their corresponding full n-grams and compute for Jaccard similarity. The similarity score is calculated as the ratio of the intersection size to the union size of the two n-gram sets. This comparison is implemented using set operations in Python.

```python
def jaccard_similarity(set1, set2):

    intersection = set1 & set2
    union = set1 | set2
    if not union:
        return 0.0
    return len(intersection) / len(union)
```

4.3.2.5 Results and Limitations

This approach consumes a large amount of storage, especially with large datasets consisting of hundreds of thousands of sentences. Since every sentence is broken down into multiple n-grams and stored in the ngrams table, data can grow rapidly, consuming significant storage and memory resources. This leads to slower response times and reduced scalability for real-time applications.

4.3.3 Bidirectional Encoder Representations from Transformers (BERT)

Bidirectional Encoder Representations from Transformers (BERT) is an open-source language model developed by Google that can capture the semantic meaning of sentences. This allows us to apply it in our tool to detect similar content. This section outlines the use of this method and the implementation of BERT with FAISS (Facebook AI Similarity Search) to perform plagiarism detection.

4.3.3.1 Method Overview

For under-resourced languages like Khmer, we utilize a multilingual BERT model, paraphrase-multilingual-MiniLM-L12-v2, which could capture the semantic meaning of Khmer text and supports cross-lingual similarity [2]. To enable efficient similarity search, we employ FAISS (Facebook AI Similarity Search), which allows for fast approximate nearest neighbor retrieval in large vector spaces, significantly reducing computation time compared to brute-force pairwise comparisons. In our implementation, we use the IndexFlatL2 index, which performs search using the L2 (Euclidean) distance [14].

4.3.3.2 Method Implementation Process Flow

To implement our plagiarism detection tool with BERT and FAISS, we divide the process into two main stages: the original document preprocessing stage and the plagiarism checking stage.

1. In the preprocessing stage, after each sentence from the original documents is cleaned and tokenized using our data preprocessing pipeline, it is encoded using a pre-trained BERT model and then indexed using FAISS for efficient similarity search. This index is saved into a model file to be reused during the plagiarism detection process.
2. In the plagiarism checking stage, when a user uploads a new document, the same data preprocessing, and BERT encoding process is applied to the new document. The saved FAISS model is then loaded, and the newly encoded vectors are searched against the index to retrieve similar sentences.

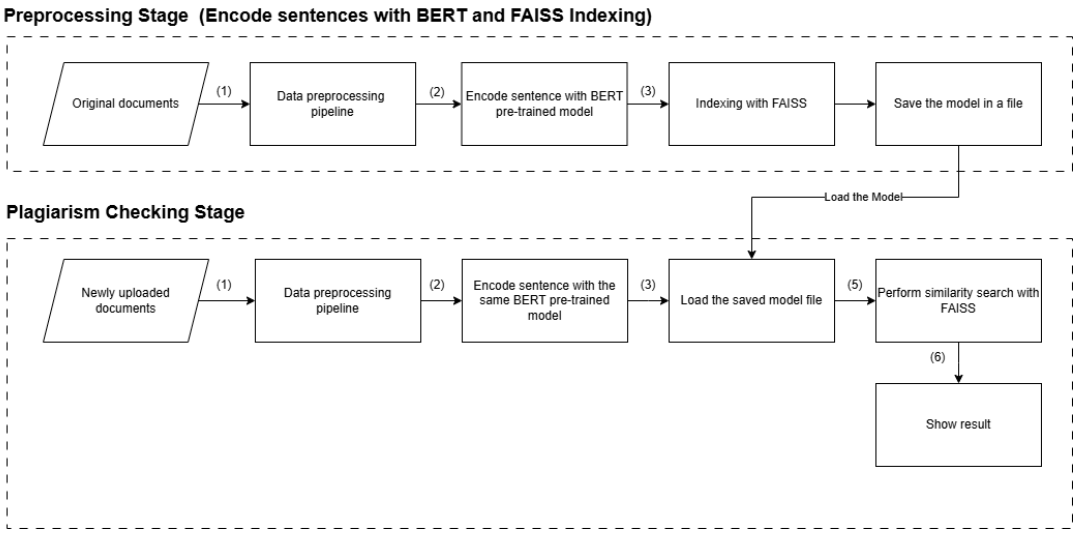


Figure 15: BERT and FAISS Pipeline Implementation

4.3.3.3 Technical Implementation

For this implementation, we use sentence transformers function from the SentenceTransformer library directly to encode the sentence to a dense vector in our Python code and FAISS library for indexing. This section outlines the encoding process using a pre-trained model from BERT and FAISS indexing.

1. Each sentence is converted into a high-dimensional dense vector using a pre-trained multilingual model called paraphrase-multilingual-MiniLM-L12-v2 from transformer, which allows the system to detect some exact matches as well as paraphrased content.

```
from sentence_transformers import SentenceTransformer

# Utilize paraphrase-multilingual-MiniLM-L12-v2 to support for Khmer language
model_MiniLM = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')

# Combine all sentences and encode with the model
embeddings = model_MiniLM.encode(sentences, convert_to_numpy=True)
```

2. These vectors are added to a FAISS index to support fast and scalable search. This index allows fast comparison between new sentences and the existing dataset by vector distance. For this implementation, we can call the FAISS library directly and add the embeddings to the index.

```
import faiss
import numpy as np

vector_dimension = embeddings.shape[1]
index = faiss.IndexFlatL2(vector_dimension)
index.add(embeddings)
```

4.3.3.4 Results and Limitations

One of the key limitation lies in the method's scalability, as frequently adding new documents to the dataset requires re-encoding the entire corpus, which becomes increasingly time-consuming as the dataset grows. Additionally, this method relies on a GPU to process high-dimensional embeddings efficiently. However, such hardware is not currently available on the MoEYS server.

4.3.4 Elasticsearch-Based Plagiarism Detection Method

This section outlines the fourth approach to plagiarism detection using Elasticsearch. Elasticsearch is widely used due to its search capabilities, making it a suitable candidate for identifying potential plagiarized content.

4.3.4.1 Method Overview

Elasticsearch is a search engine that supports fast and efficient full-text search. It is designed to store, index, and search large volumes of text data effectively. Like the N-gram and Jaccard similarity method described earlier, Elasticsearch also uses an inverted index—a data structure that maps each term in the dataset to the list of documents where it appears [15]. For ranking results, Elasticsearch uses the BM25 (Best Matching 25) algorithm, which builds upon TF-IDF by also considering document length. This slightly favors shorter documents and helps prevent longer documents from gaining an unfair advantage simply by repeating terms more frequently [16].

4.3.4.2 Method Implementation Process Flow

To implement our plagiarism detection tool with Elasticsearch, we divide the process into two main stages: the original dataset preprocessing stage and the plagiarism checking stage.

1. In the preprocessing stage, each sentence from the original documents is cleaned and tokenized using our data preprocessing pipeline. These processed sentences are then stored in an Elasticsearch index, enabling efficient retrieval and full-text search capabilities during plagiarism detection.
2. In the plagiarism checking stage, when a user uploads a new document, it will go through the same cleaning and tokenization process using our data preprocessing pipeline. The system then queries the Elasticsearch index to retrieve sentences that are textually similar to the uploaded content. If any matching results are above similarity threshold, they are flagged and presented as potential plagiarism cases.

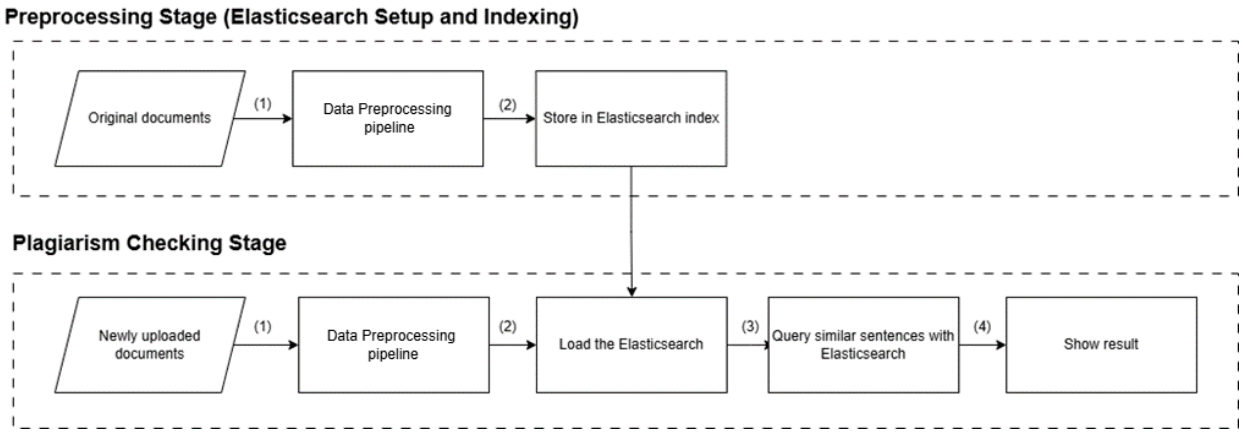


Figure 16: Elasticsearch Pipeline Implementation

4.3.4.3 Technical Implementation

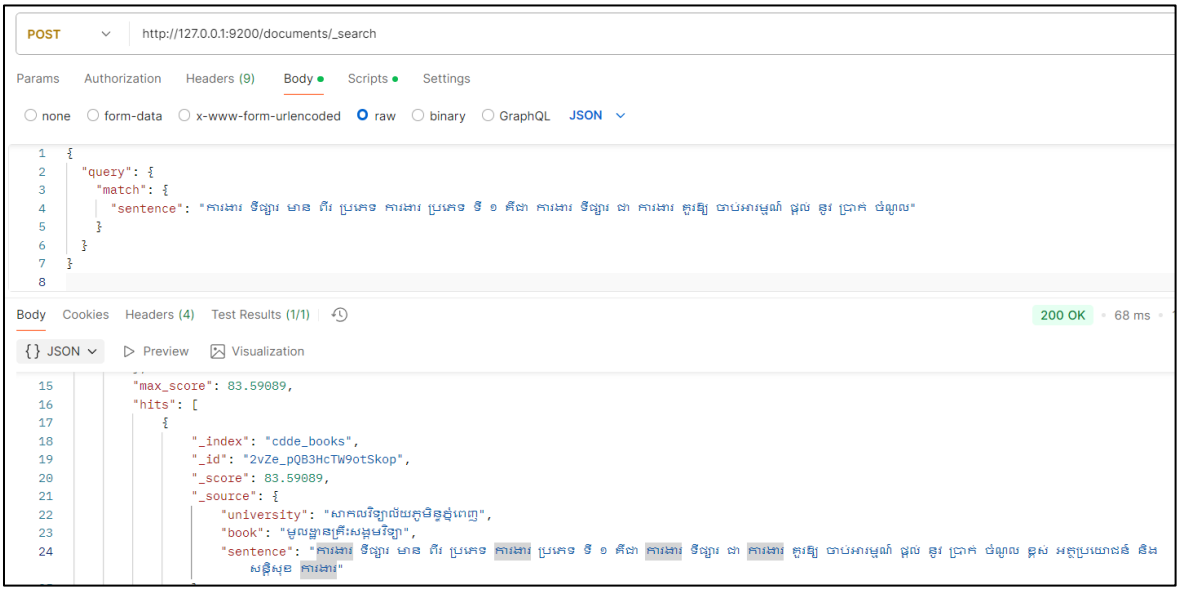
The Elasticsearch method involves configuring a full-text search engine that can index and query large volumes of sentence-level data. This section outlines how we setup the Elasticsearch, and use for indexing and querying.

1. Docker was used to deploy Elasticsearch, which was configured to run on port 9200 and made accessible via its RESTful API. The application utilized the official Python Elasticsearch client to communicate with the containerized instance for both indexing and query operations.

2. To enable efficient search, a custom mapping was first defined in Elasticsearch to specify the structure of the indexed documents. This mapping included fields such as title, authors, publication year, university, and sentence. Once the index was created with the defined mapping, documents adhering to this structure were inserted into the system.

```
# Define mapping with all required fields
mapping = {
    "mappings": {
        "properties": {
            "title": {"type": "text"},
            "authors": {"type": "text"},
            "publish_year": {"type": "integer"},
            "university": {"type": "keyword"},
            "sentence": {
                "type": "text",
                "fields": {
                    "keyword": {"type": "keyword"}  # Enables exact search
                }
            }
        }
    }
}
# Create the index with the mapping
response = requests.put(
    f"{ELASTICSEARCH_URL}/{INDEX_NAME}",
    headers={"Content-Type": "application/json"},
    data=json.dumps(mapping)
)
```

3. Elasticsearch analyzes the text from our queries and returns the most relevant matches based on scoring result.



4.3.4.4 Results and Limitations

Elasticsearch performed well both speed and accuracy. It can find all the identical and similar sentences in the database in short period of time. Even when the input sentences had small changes, such as one or two extra or missing words, Elasticsearch still returned similar results. This shows that it can handle both exact matches and slightly different sentences, making it a strong choice for our plagiarism detection tool requirement.

5. Method Evaluation and Selection

This section presents a comparative analysis of the plagiarism detection methods implemented in the system. The goal is to identify the most effective approach based on detection accuracy, execution time, memory usage, and scalability. The first subsection outlines the evaluation methodology and testing criteria, while the second provides a justification for the final method selection based on observed results.

5.1 Evaluation

To evaluate the effectiveness of each plagiarism detection method, we tested whether the system could successfully return all expected matches for a set of known plagiarized sentences. We randomly selected 600 sentences from the 890 indexed documents and slightly modified them by inserting, deleting, or substituting words, while still preserving the original content and sentence structure.

Each method including TF-IDF with cosine similarity, N-gram with Jaccard similarity, Elasticsearch, and BERT embeddings with FAISS was evaluated based on its ability to accurately retrieve all 600 sentences. The comparison focuses on accuracy, execution time, memory usage, and scalability. The results are summarized in the table below.

Methods	Score	Time Execution (second)	Peak Memory Usage (MB)	Scalability
TF-IDF & Cosine	92.55	20.58	6984.18	Inefficient for frequent data insertions, deletions, or updates due to the need to recompute the entire TF-IDF matrix.
Ngrams and Jaccard (PostgreSQL inverted index)	92.83	2,546	15.88	Not scalable for large datasets, as n-gram growth significantly increases memory and slows down query performance.
paraphrase-multilingual-miniLM-L12-v2 BERT	67.83	4.37	7.41	Frequent data updates require intensive memory and computation to rebuild.
Elasticsearch & RapidFuzz	98.17	31.97	9.34	Elasticsearch allows parallel search and indexing for large datasets.

Table 1: Methods Comparison

5.2 Final Method Selection

Based on the evaluation results, Elasticsearch offers the best overall balance between accuracy, performance, memory efficiency, and scalability. It successfully returned all exact matches and similar sentences while maintaining stable performance and using less memory. Although it is not faster than TF-IDF or BERT in some cases, Elasticsearch supports incremental updates and parallel indexing, making it well-suited for large-scale and continuously growing datasets. Given these advantages, Elasticsearch was selected as the final method for our plagiarism detection web application.

6. System Architecture and Workflow

This section outlines the overall architecture of the proposed plagiarism detection system, with a focus on how different technology components interact to support both uploading document to the database and plagiarism detection. It describes the technical components involved in the backend infrastructure and explains the key workflows from system input to output. The following subsections provide an overview of the system's core components and describe two key backend workflows: one for indexing documents and another for detecting plagiarism.

6.1 System Architecture

The proposed system is composed of multiple technology components, including React.js, Flask API, Redis, MinIO file storage, Google Cloud Vision OCR, and Python scripts for the data preprocessing pipeline. These components work together to prepare documents for indexing and querying in Elasticsearch. Figure 16 illustrates how these components interact throughout the system's workflow.

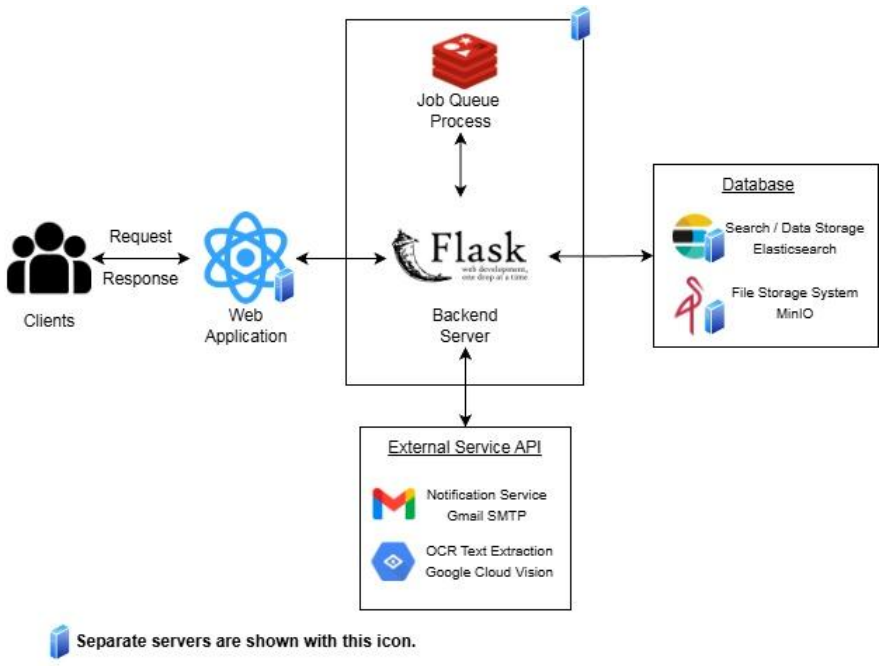


Figure 17: System Architecture

This architecture supports asynchronous processing through Redis, efficient file management via MinIO, OCR text extraction, text preprocessing with Python, and scalable similarity search through Elasticsearch. The design enables the system to scale with increasing document and supports the future integration of additional detection methods.

6.2 Document Uploading Workflow

This section outlines the process of uploading more documents to MinIO file storage system and indexing their content in Elasticsearch. The workflow handles both PDF and Microsoft Word files by extracting their text, preprocessing the content, and storing the results in an Elasticsearch index. The diagram below illustrates the process starting from document datasets are inserted until users receive its status.

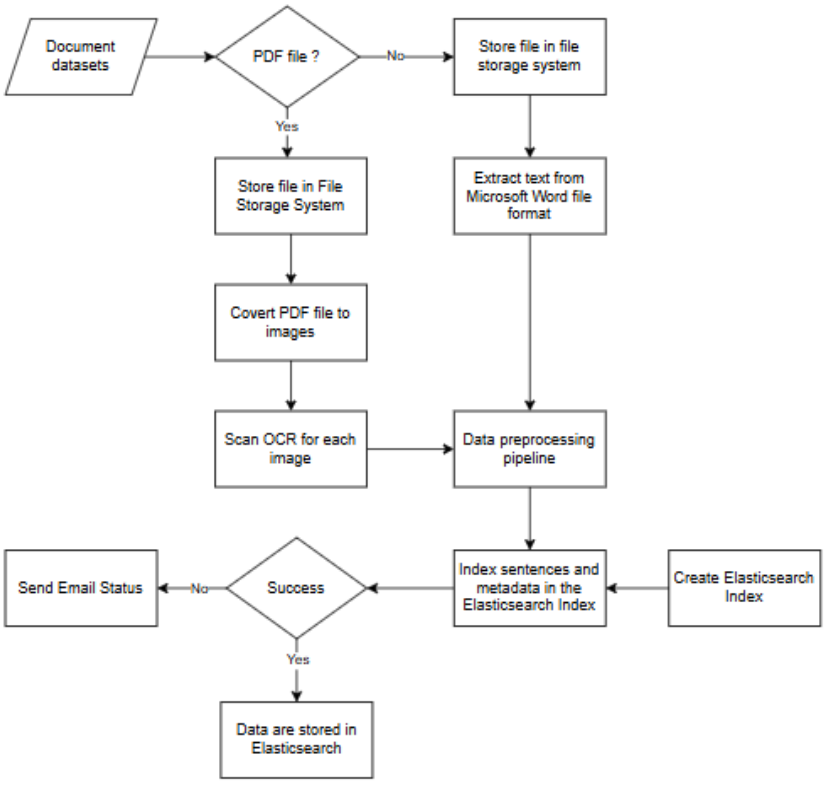


Figure 18: Documents Indexing Workflow

As shown in the diagram above, the workflow begins with the input of document datasets. Depending on the file type, the system either extracts text from Microsoft Word files directly or applies OCR to PDF files after converting them into images. The extracted text is then passed through a data preprocessing pipeline before being indexed into Elasticsearch. Once indexing is successful, the system stores the processed data and sends a status notification via email.

6.3 Plagiarism Detection Workflow

This section outlines the workflow used to detect plagiarism in user-submitted documents by comparing their content against previously indexed documents in Elasticsearch. The system supports both PDF and Microsoft Word file formats, ensuring flexibility and compatibility with common academic materials. After extracting and preprocessing the text, each sentence is matched against the existing Elasticsearch index. Sentences with a similarity score above a predefined threshold are considered potential matches and compiled into a report.

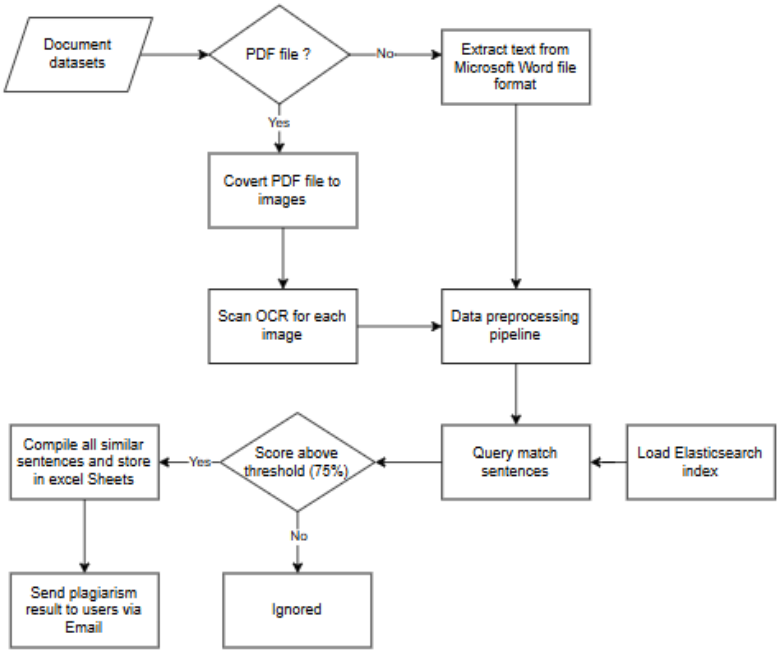


Figure 19: Plagiarism Detection Workflow

As shown in the diagram above, the plagiarism detection process begins with the upload of a document dataset. The system first checks whether the file is in PDF format. If it is, the PDF is converted into images and processed through OCR to extract text. If the file is a Microsoft Word

document, the text is extracted directly. The raw text is then passed through a data preprocessing pipeline. Once preprocessed, the sentences are queried against the Elasticsearch index. If the similarity score is above the threshold (75%), the matched sentence is retained. All matched sentences are compiled into an Excel report and send to the users via email, completing the plagiarism detection process.

7. Web Application Implementation

This section outlines the implementation of the web application, which was developed using Flask for the backend API and React for the frontend interface. The system enables users to interact easily with the plagiarism detection tool through a user interface. The application supports three core functionalities: plagiarism checking, indexing new documents into MinIO file storage system and Elasticsearch, and viewing and managing stored documents.

7.1 Plagiarism Detection Webpage

The plagiarism detection webpage allows users to drag and drop multiple documents for analysis, supporting two file formats: PDF and Microsoft Word (.docx). Users are required to enter their email address to receive the plagiarism detection results once processing is complete.

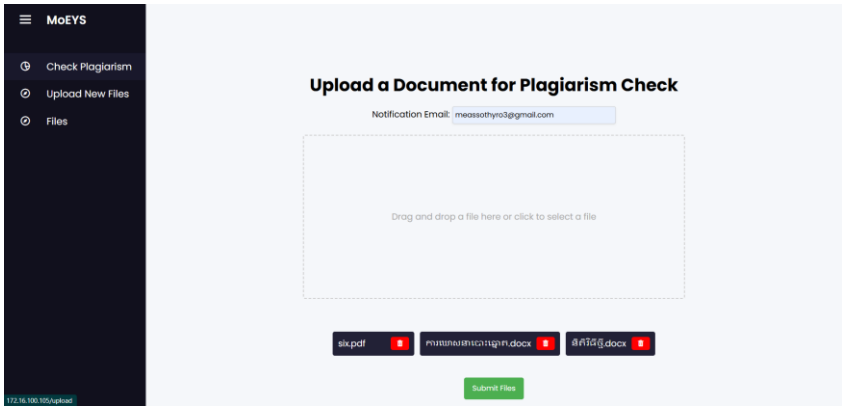


Figure 20: Plagiarism Detection Webpage

After users upload the files, they want to check for plagiarism, they can click the "Upload" button. The data is then packaged into a FormData object and sent to the Flask backend server

via an HTTPS POST request using the Fetch API. The backend receives the files and email input from users for asynchronous processing.

To handle high-volume uploads during peak hours, the backend is integrated with a Redis server to queue document processing tasks. Each job is queued and processed one at a time. A Redis dashboard, accessible via port 9181, allows administrators to monitor the job queue, track processing status, and identify failed tasks, as shown in the following figure.

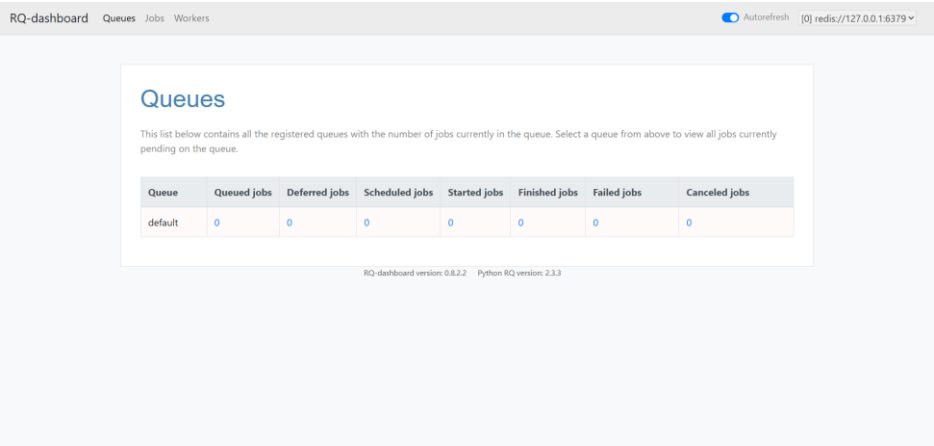
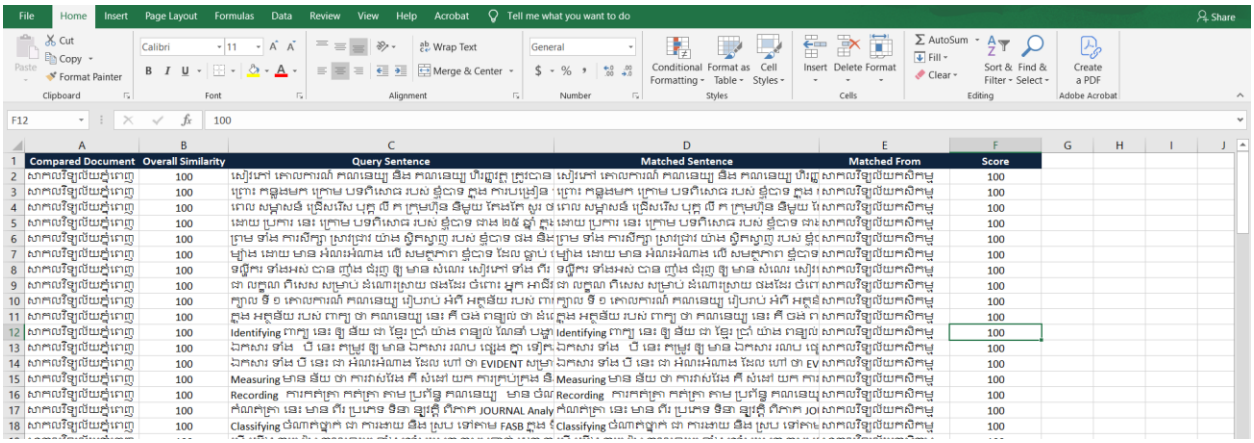


Figure 21: Redis Queue Dashboard Webpage

Once a job reaches the front of the queue, the plagiarism detection process begins by identifying the file type. If the file is a PDF, it is concurrently converted into images and then processed with OCR to extract text. The extracted text undergoes preprocessing, as described in Section 4.2.3. After preprocessing, each sentence in the cleaned data is queried against the Elasticsearch index. If the similarity score exceeds a defined threshold, the sentence is retained and compiled into an Excel report using the ExcelWriter Python library. The report includes columns such as the sentence from the uploaded document, the matched sentence found in the database, the university name of the original source, the name of the original document that was plagiarized, and the overall similarity score.



Compared Document	Overall Similarity	Query Sentence	Matched Sentence	Matched From	Score
សាកលវិទ្យាល័យភូមិន្ទកសិកម្ម	100	សៀវភៅ គោលការណ៍ គណនេយ្យ និង គណនេយ្យ ហិរញ្ញវត្ថុ ច្បាប់	សៀវភៅ គោលការណ៍ គណនេយ្យ និង គណនេយ្យ ហិរញ្ញ	សាកលវិទ្យាល័យកសិកម្ម	100
សាកលវិទ្យាល័យភូមិន្ទកសិកម្ម	100	ព្រោះ កន្លងមក ក្រោម បទពិសោធ របស់ ខ្ញុំបាទ ក្តី ការបម្រើ	ព្រោះ កន្លងមក ក្រោម បទពិសោធ របស់ ខ្ញុំបាទ ក្តី	សាកលវិទ្យាល័យកសិកម្ម	100
សាកលវិទ្យាល័យភូមិន្ទកសិកម្ម	100	នោះ សម្ពាសន៍ ជ្រើសរើស បុគ្គ ល ក ក្រុមហ៊ុន មិមួយ តែងតែ សូ ម នោះ សម្ពាសន៍ ជ្រើសរើស បុគ្គ ល ក ក្រុមហ៊ុន មិមួយ		សាកលវិទ្យាល័យកសិកម្ម	100
សាកលវិទ្យាល័យភូមិន្ទកសិកម្ម	100	ដោយ ប្រការ នេះ ក្រោម បទពិសោធ របស់ ខ្ញុំបាទ ជាង ២៥ ឆ្នាំ ក្តី	ដោយ ប្រការ នេះ ក្រោម បទពិសោធ របស់ ខ្ញុំបាទ ជា	សាកលវិទ្យាល័យកសិកម្ម	100
សាកលវិទ្យាល័យភូមិន្ទកសិកម្ម	100	ព្រម ទាំង ការសិក្សា ស្រាវជ្រាវ យ៉ាង ស្វិតស្វាញ របស់ ខ្ញុំបាទ ផង និង ព្រម ទាំង ការសិក្សា ស្រាវជ្រាវ យ៉ាង ស្វិតស្វាញ របស់ ខ្ញុំ		សាកលវិទ្យាល័យកសិកម្ម	100
សាកលវិទ្យាល័យភូមិន្ទកសិកម្ម	100	ម្យ៉ាង ដោយ មាន អំណរអំណាង លើ សមត្ថភាព ខ្ញុំបាទ ដែល ធ្លាប់ ម្យ៉ាង ដោយ មាន អំណរអំណាង លើ សមត្ថភាព ខ្ញុំបាទ		សាកលវិទ្យាល័យកសិកម្ម	100
សាកលវិទ្យាល័យភូមិន្ទកសិកម្ម	100	ទណ្ឌិត ទាំងអស់ បាន ញ៉ែង ជំរុញ ឲ្យ មាន សំណេ សៀវភៅ ទាំង ពីរ ទណ្ឌិត ទាំងអស់ បាន ញ៉ែង ជំរុញ ឲ្យ មាន សំណេ សៀវ		សាកលវិទ្យាល័យកសិកម្ម	100
សាកលវិទ្យាល័យភូមិន្ទកសិកម្ម	100	ជា លក្ខណ ពិសេស សម្រាប់ ដំណោះស្រាយ ផងដែរ ចំពោះ អ្នក អាជីវ ជា លក្ខណ ពិសេស សម្រាប់ ដំណោះស្រាយ ផងដែរ ចំពោ		សាកលវិទ្យាល័យកសិកម្ម	100
សាកលវិទ្យាល័យភូមិន្ទកសិកម្ម	100	ក្បាល ទី ១ គោលការណ៍ គណនេយ្យ ហិរិយាប អំពី អត្ថន័យ របស់ ពាក្យាល ទី ១ គោលការណ៍ គណនេយ្យ ហិរិយាប អំពី អត្ថន័		សាកលវិទ្យាល័យកសិកម្ម	100
សាកលវិទ្យាល័យភូមិន្ទកសិកម្ម	100	ក្តីង អត្ថន័យ របស់ ពាក្យ ថា គណនេយ្យ នេះ គឺ ចង់ ពន្យល់ ថា ដំណើង អត្ថន័យ របស់ ពាក្យ ថា គណនេយ្យ នេះ គឺ ចង់ ព		សាកលវិទ្យាល័យកសិកម្ម	100
សាកលវិទ្យាល័យភូមិន្ទកសិកម្ម	100	Identifying ពាក្យ នេះ ឲ្យ ឱយ ដា ឡែះ ប្រាំ យ៉ាង ពន្យល់ ណែនាំ បង្ហា	Identifying ពាក្យ នេះ ឲ្យ ឱយ ដា ឡែះ ប្រាំ យ៉ាង ពន្យល់	សាកលវិទ្យាល័យកសិកម្ម	100
សាកលវិទ្យាល័យភូមិន្ទកសិកម្ម	100	ឯកសារ ទាំង ពី នេះ ពាម្រាះ ឲ្យ មាន ឯកសារ រលាប ឈ្លេង ក្តា ទៀត ឯកសារ ទាំង ពី នេះ ពាម្រាះ ឲ្យ មាន ឯកសារ រលាប ផ្តេ		សាកលវិទ្យាល័យកសិកម្ម	100
សាកលវិទ្យាល័យភូមិន្ទកសិកម្ម	100	ឯកសារ ទាំង ពី នេះ ជា អំណរអំណាង ដែល ហៅ ថា EVIDENT សម្ភា ឯកសារ ទាំង ពី នេះ ជា អំណរអំណាង ដែល ហៅ ថា EV		សាកលវិទ្យាល័យកសិកម្ម	100
សាកលវិទ្យាល័យភូមិន្ទកសិកម្ម	100	Measuring មាន ឱយ ថា ការសង្កេត គឺ សំអៅ យក ការប្រៀបក្រែង ធ្វើ	Measuring មាន ឱយ ថា ការសង្កេត គឺ សំអៅ យក ការ	សាកលវិទ្យាល័យកសិកម្ម	100
សាកលវិទ្យាល័យភូមិន្ទកសិកម្ម	100	Recording ការកត់ត្រា កត់ត្រា តាម ប្រព័ន្ធ គណនេយ្យ មាន ជំរុ	Recording ការកត់ត្រា កត់ត្រា តាម ប្រព័ន្ធ គណនេយ្យ	សាកលវិទ្យាល័យកសិកម្ម	100
សាកលវិទ្យាល័យភូមិន្ទកសិកម្ម	100	ការណ៍ត្រា នេះ មាន ពីរ ប្រភេទ ទឹកា ឡូត្តិ ពីភាគ JOURNAL Analy	ការណ៍ត្រា នេះ មាន ពីរ ប្រភេទ ទឹកា ឡូត្តិ ពីភាគ JO	សាកលវិទ្យាល័យកសិកម្ម	100
សាកលវិទ្យាល័យភូមិន្ទកសិកម្ម	100	Classifying ដំណាក់ថ្នាក់ ជា ការងាយ និង ស្រប ទៅតាម FASB ក្តី	Classifying ដំណាក់ថ្នាក់ ជា ការងាយ និង ស្រប ទៅតា	សាកលវិទ្យាល័យកសិកម្ម	100

Figure 22: Results Report in Excel

In addition to the Excel report, a summary certificate is also sent, which includes the overall similarity score as well as the percentage of content plagiarized from each matched document. To generate this certificate, the WeasyPrint library is used to render an HTML template into a formatted PDF that highlights document names with high similarity.

បង្ហាញលទ្ធផលនៃការត្រួតពិនិត្យអត្ថបទ

ការស្រដៀងគ្នាថ្នាក់ខ្ពស់បំផុត: 100.00%

សៀវភៅដែលបានប្រៀបធៀប	ការស្រដៀងគ្នា (%)	សាកលវិទ្យាល័យ
one	100.00	សាកលវិទ្យាល័យភូមិន្ទកសិកម្ម
វិធីសាស្ត្រស្រាវជ្រាវសម្រាប់សេដ្ឋកិច្ចកសិកម្ម	3.23	សាកលវិទ្យាល័យភូមិន្ទកសិកម្ម
គណនេយ្យហិរញ្ញវត្ថុ ភាគ១	3.23	សាកលវិទ្យាល័យជាតិគ្រប់គ្រង
ការអនុវត្តគណនេយ្យហិរញ្ញវត្ថុ	3.23	សាកលវិទ្យាល័យជាស៊ីមកំចាយមារ
មូលដ្ឋានគ្រឹះនៃគោលការណ៍គណនេយ្យ	3.23	សាកលវិទ្យាល័យជាស៊ីមកំចាយមារ
ការគ្រោងបង្កើតផលិតផល	3.23	សាកលវិទ្យាល័យជាស៊ីមកំចាយមារ
ប្រព័ន្ធស្រោចស្រពក្នុងកសិដ្ឋាន	3.23	សាកលវិទ្យាល័យភូមិន្ទកសិកម្ម

Figure 23: Plagiarism Result Certificate

After excel report and certificate are generated. They are emailed to the user using the smtplib, EmailMessage, and MIMEText from Python libraries.

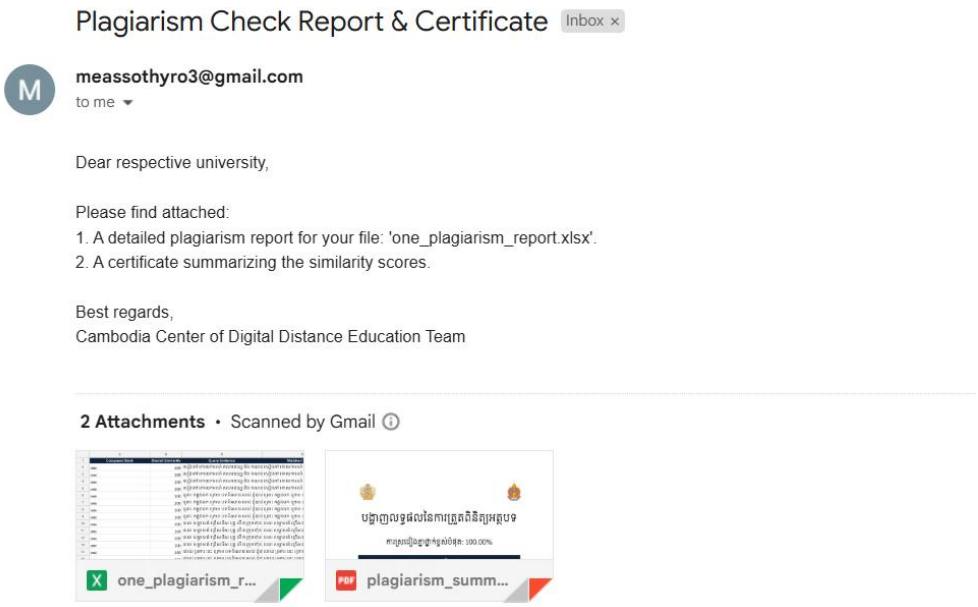


Figure 24: Plagiarism Detection Results Sent Via Email

7.2 Indexing and Uploading Documents Webpage

This webpage allows users to drag and drop multiple documents for uploading into both the Elasticsearch index and the file storage system. This functionality supports improved detection accuracy over time as the dataset grows. The drag-and-drop area supports two file formats: PDF and Microsoft Word (.docx). Users are required to select the university they want to upload the documents to, and also enter their email address to receive a status update once the upload process to both MinIO and Elasticsearch is complete.

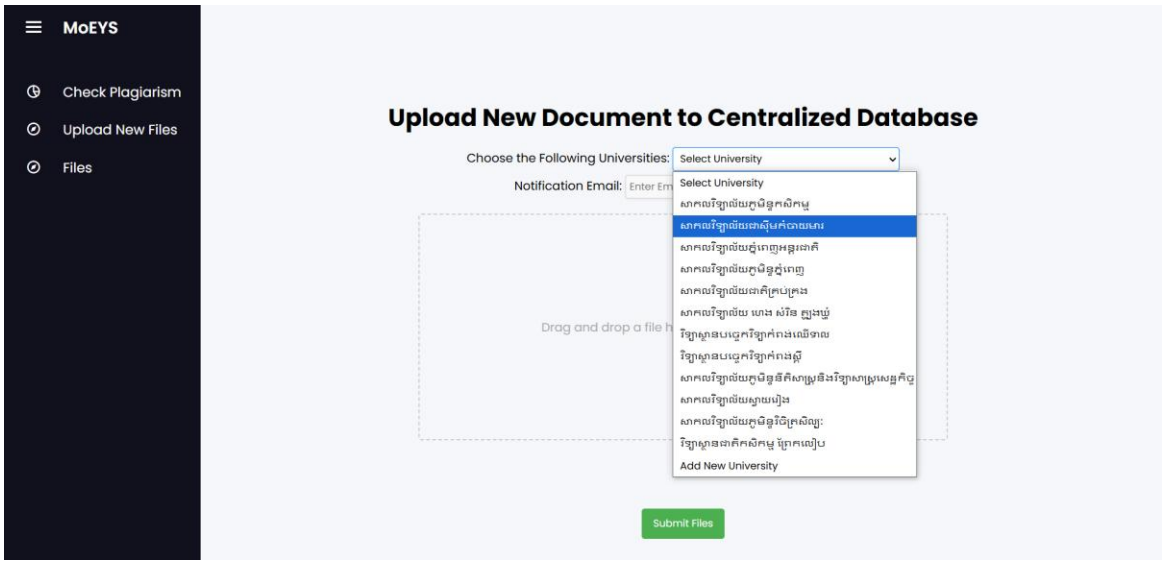


Figure 25: Indexing and Uploading Documents Webpage

After selecting the target university and enter their email, users can drop the files into the designated area and click the "Upload" button to initiate the upload. The selected files and user input are then packaged into a FormData object and sent to the Flask backend server via an HTTPS POST request using the Fetch API. Once received, the backend uses the MinIO client library—along with the configured hostname and access credentials—to store the files in the appropriate bucket and folder path. These uploaded files can be viewed via the MinIO web dashboard, accessible on port 9001, as shown in the following figure.

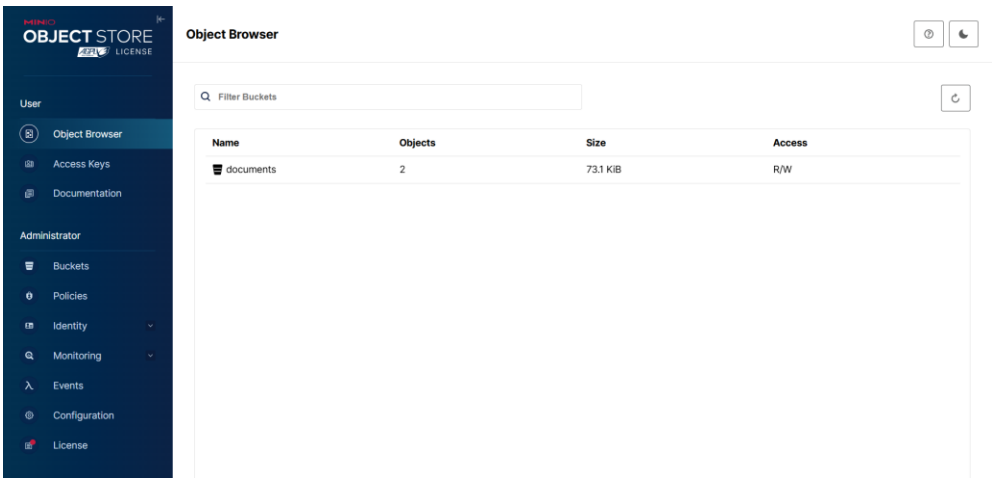


Figure 26: MinIO Webpage

To index the data in Elasticsearch, the backend preprocesses the extracted text and formats it according to the predefined index mapping. The *bulk()* function from the Elasticsearch library is then used to efficiently insert the new data. After both Elasticsearch indexing and MinIO storage are successfully completed, the smtplib library is triggered. The *EmailMessage* and *MIMEText* Python libraries are used to customize the email subject, body, and content, which are then sent to notify the user whether the upload was successful.

7.3 File Storage System Webpage

This webpage allows users to view all documents stored in the file storage system and delete them without needing to access the MinIO dashboard directly. It displays all available storage locations where documents have been uploaded. Additionally, users can download a document by clicking on its name or delete it by clicking the delete button, which removes the file not only from MinIO but also from the corresponding Elasticsearch index.

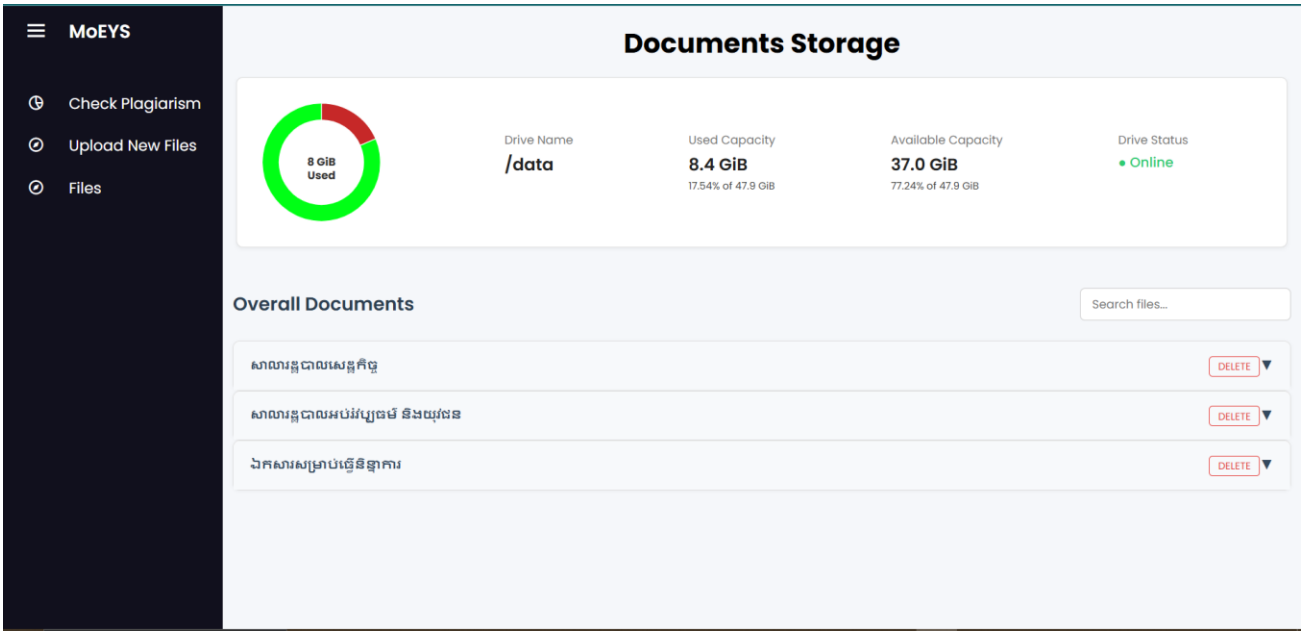


Figure 27: MinIO Custom Storage Webpage

8. Current Results and Limitations

The current application is designed to perform database-based plagiarism detection, allowing users to upload thousands of documents and check for similarities against content stored in the system's large internal database. It is capable of retrieving all exact matches as well as detecting some sentences that have been slightly modified.

One significant limitation lies in the OCR extraction process for the Khmer language. The accuracy of existing OCR tools remains relatively low for low-resolution documents, which directly affects the quality of extracted text. Since the platform relies heavily on OCR to process PDF files, improving Khmer OCR accuracy is essential to enhancing the system’s overall reliability and detection performance.

9. User Feedback

During testing and review, staff at the Ministry of Education, Youth and Sport commented that "*The web application is efficient and fast. It allows us to upload documents and find similar sentences easily.*" However, they also observed that "*However, uploading large PDF files format for either plagiarism detection process or into a file storage system takes noticeably longer compared to Microsoft Word documents file format.*" This feedback highlights a key performance difference due to the need for Optical Character Recognition (OCR) when handling PDF files, whereas Microsoft Word files provide faster results.

10. Future Plan

For future development, we aim to improve the accuracy of Khmer OCR, which remains a critical challenge for reliable text extraction. We plan to conduct further research into OCR optimization techniques, such as custom model training, image preprocessing, and post-OCR correction. Enhancing OCR performance will significantly strengthen the quality of extracted text and, in turn, the accuracy of plagiarism detection.

Additionally, we plan to extend the system’s capabilities to support plagiarism detection from external online sources. This will involve enabling users to query and detect plagiarized content directly from publicly available websites and internet resources, thereby expanding coverage beyond the existing internal database and improving the overall effectiveness of the detection system.

11. Conclusion

This project presents the design, and the implementation of a Khmer-language plagiarism detection system aimed at addressing the lack of digital tools for academic integrity in Cambodia. This project explores multiple detection methods—including TF-IDF with cosine similarity, N-gram with Jaccard similarity stored in PostgreSQL inverted index, BERT-based semantic matching, and Elasticsearch-based retrieval—to compare uploaded documents against a growing database of academic texts. Each method was evaluated for accuracy, scalability, and performance. The final system selects Elasticsearch to integrate with the web application, allowing users to interact easily.

The application allows users to upload Microsoft Word and PDF files, processes them through a preprocessing pipeline, and performs similarity checks using a web-based interface. Feedback from the Ministry of Education, Youth and Sport confirmed the system's effectiveness in identifying matching content, although OCR processing time and accuracy for PDF files remains a limitation. The system is currently deployed on a self-hosted server within the Ministry’s infrastructure, enabling secure access for internal users.

In conclusion, this tool represents an important step toward modernizing academic quality control in Cambodia. While the current implementation focuses on database-based detection, future development will explore extending the system to query external web content, further improving its ability to detect paraphrased and copied material from online sources.

Bibliography

[1] T. Phallavattana, "Plagiarism Among University Students: Causes, Consquences, and Recommendation," Cambodia Education Forum, 18 September 2023. [Online]. Available: https://cefcambodia.com/2023/09/18/plagiarism-among-university-students-causes-consequences-and-recommendations/.

[2] A. M. T. G. a. C. D. Karen Avetisyan, "A Simple and Effective Method of Cross - Lingual Plagiarism Detection," *Research Square,* 2023.

[3] N. Thuon, "Khmer Semantic Search Engine," *Digital Information and Document Retrieval,* 2024`.

[4] S. Canny, "python-docx Documentation," 2013. [Online]. Available: https://python-docx.readthedocs.io/en/latest/.

[5] G. Developer, "Tesseract Open Source OCR Engine (main repository)," Google, 2005. [Online]. Available: https://github.com/tesseract-ocr/tessdata/blob/main/khm.traineddata.

[6] S. Ratanak, "Khmer Analysis Plugin for Elasticsearch," Khmerlang, 28 August 2022. [Online]. Available: https://www.khmerlang.com/posts/6.

[7] M. Durdin, "khmer normalizer," Github, 12 August 2024. [Online]. Available: https://github.com/sillsdev/khmer-normalizer.

[8] Seanghay, "A fast Khmer word segmentation toolkit," pypi.org, 10 February 2025. [Online]. Available: https://pypi.org/project/khmercut/.

[9] S. Ratanak, "Khmer spellcheck correction," Khmerlang, 2022. [Online]. Available: https://khmerlang.com/tools/khmer-spelling-check.

[10] T. Nimol, "KSWv2-Khmer-Stop-Word-based-Dictionary-for-Keyword-Extraction," Github, 19 October 2024. [Online]. Available: https://github.com/back-kh/KSWv2-Khmer-Stop-Word-based-Dictionary-for-Keyword-Extraction.

[11] M. U. Hutabarat, "Comparing Text Documents Using TF-IDF and Cosine Similarity in Python," Medium, 17 December 2023. [Online]. Available: https://medium.com/@mifthulyn07/comparing-text-documents-using-tf-idf-and-cosine-similarity-in-python-311863c74b2c.

[12] P. Sharma, "How to Save and Load Machine Learning Models in Python Using Joblib library," [Online]. Available: https://www.analyticsvidhya.com/blog/2023/02/how-to-save-and-load-machine-learning-models-in-python-using-joblib-library/. [Accessed 04 April 2025].

[13] N. E. Diana, "Measuring Performance of N-Gram and Jaccard Similarity Metrics in Document Plagiarism Application," *Researchgate,* 2019.

[14] Pinecone, "Introduction to Facebook AI Similarity Search," Pinecone, 2022. [Online]. Available: https://www.pinecone.io/learn/series/faiss/faiss-tutorial/.

[15] A. Brasetvik, "Elasticsearch from the Bottom Up," 17 September 2023. [Online]. Available: https://www.elastic.co/blog/found-elasticsearch-from-the-bottom-up.

[16] S. Connelly, "Practical BM25 - Part2: The BM25 Algorithm and its variables," 19 April 2018. [Online]. Available: https://www.elastic.co/blog/practical-bm25-part-2-the-bm25-algorithm-and-its-variables.

[17] L. P. Sirivath, "Culture ministry wanrs against plagiarism as Berne Convention set to come into force," Phnom Penh Post, 25 February 2022. [Online]. Available: https://www.phnompenhpost.com/national/culture-ministry-warns-against-plagiarism-berne-convention-set-come-force.

[18] M. Potthast, "Cross-Language plagiarism detection," *Language Resources and Evaluation,* 2011.

[19] S. Ratanak, "How to Khmer OCR with Tesseract in Python," Khmerlang, 1 June 2022. [Online]. Available: https://www.khmerlang.com/posts/5.

[20] GeeksforGeeks, "Understanding TF-IDF (Term Frequency - Inverse Document Frequency)," 07 February 2025. [Online]. Available: https://www.geeksforgeeks.org/understanding-tf-idf-term-frequency-inverse-document-frequency/.

[21] F. Fusco, "an Efficient all MLP Architecture for Language," *Research gate,* p. 3, February 2022.

[22] Pinecone, "Locality Sensitive Hashing (LSH): The Illustrated Guide," Pinecone, 2022. [Online]. Available: https://www.pinecone.io/learn/series/faiss/locality-sensitive-hashing/.

[23] M. Documentation, "Install and Deploy MinIO," 2022. [Online]. Available: https://min.io/docs/minio/linux/operations/installation.html.

[24] C. Le, "Setting Up Redis as a Message Queue: A Step-by-Step Guide," DEV community, 10 September 2024. [Online]. Available: https://dev.to/chanh_le/setting-up-redis-as-a-message-queue-a-step-by-step-guide-5gj0.